

C言語プログラミング演習問題集

基礎問題からチャレンジ問題まで段階的に取り組む

C言語学習教材プロジェクト

目次

第1章:C言語入門の演習問題	3
第2章:基本構文の演習問題	6
第3章:データ型と変数の演習問題	13
第4章:演算子の演習問題	17
第5章:条件分岐の演習問題	22
第6章:繰り返し処理の演習問題	29
第7章:配列の演習問題	35
第8章:ポインタの演習問題	42
第9章:配列とポインタの演習問題	47
第10章:文字列の演習問題	52
第11章:関数の演習問題	58
第12章:ビット操作の演習問題	68
第13章:構造体の演習問題	74
第14章:関数ポインタの演習問題	81
第15章:分割コンパイルと発展技術の演習問題	88
第16章:C23の機能の演習問題	95
補章2:開発環境の演習問題	102

第1章:C言語入門の演習問題

演習の目的

この演習を通して、C言語の開発環境が正しく動作するか確認し、簡単なプログラムを作成・実行できるようになります。

基礎問題

問題1-1:Hello Worldの実行

examples/hello_world.c をコンパイルして実行してください。

```
# コンパイル

gcc examples/hello_world.c -o hello_world

# 実行

./hello_world
```

期待される出力

```
Hello, World!
```

問題1-2:自分でHello Worldを書く

新しいファイル solutions/ex1_2_my_hello.c を作成し、Hello Worldプログラムを自分で書いてください。

ヒント

- `#include <stdio.h>` を忘れずに
- `main` 関数の中に `printf` を書く
- 最後に `return 0;` を書く

問題1-3:日本語メッセージの表示

solutions/ex1_3_japanese.c を作成し、以下のメッセージを表示するプログラムを書いてください。

表示するメッセージ

```
こんにちは。
C言語の世界へようこそ。
プログラミングを始めます。
```

ヒント: `printf` を3回使うか、`\n` で改行します。

応用問題

問題1-4:簡単な自己紹介プログラム

solutions/ex1_4_introduction.c を作成し、自己紹介を表示するプログラムを書いてください。

要件

- 名前を表示
- 好きなものを1つ表示
- C言語を学ぶ理由を表示

実行例

```
=== 自己紹介 ===  
名前: 山田太郎  
好きなもの: プログラミング  
C言語を学ぶ理由: ゲームを作りたいから。
```

問題1-5:数字を使った挨拶

solutions/ex1_5_greeting_with_number.c を作成し、名前と年齢を入力して挨拶するプログラムを書いてください。

実行例

```
お名前は[]: 花子  
年齢は[]: 20  
こんにちは、花子さん (20歳)。
```

ヒント

```
char name[50];  
int age;  
scanf("%s", name);  
scanf("%d", &age); /* 数字の入力には & が必要 */
```

チャレンジ問題

問題1-6:簡単な計算機

solutions/ex1_6_calculator.c を作成し、2つの数を足し算するプログラムを書いてください。

実行例

```
簡単な計算機  
1つ目の数: 10  
2つ目の数: 25  
10 + 25 = 35
```

問題1-7:ASCIIアート

solutions/ex1_7_ascii_art.c を作成し、簡単なASCIIアートを表示してください。

例

```
  /\_/\n (  o.o  )\n  > ^ <
```

または自分で好きな絵を作ってもOKです。

提出形式

各問題の回答は以下のファイルに記述してください:

- 問題1-2:solutions/ex1_2_my_hello.c
- 問題1-3:solutions/ex1_3_japanese.c
- 問題1-4:solutions/ex1_4_introduction.c
- 問題1-5:solutions/ex1_5_greeting_with_number.c
- 問題1-6:solutions/ex1_6_calculator.c
- 問題1-7:solutions/ex1_7_ascii_art.c

ヒント集

printfの使い方

```
printf("Display text\n");      /* 文字だけ */
printf("Name: %s\n", name);    /* 文字列を埋め込む */
printf("Number: %d\n", num);   /* 数字を埋め込む */
```

scanfの使い方

```
scanf("%s", name);            /* 文字列の入力（&不要） */
scanf("%d", &age);            /* 数字の入力（&必要） */
```

第2章:基本構文の演習問題

基礎問題

問題2-1:基本的な出力

プログラムを作成して以下を出力してください。

```
こんにちは、C言語の世界へ。  
私の名前は[あなたの名前]です。  
今日からプログラミングを始めます。
```

要求事項

- printf関数を使用すること
- 適切なエスケープシーケンスを使用すること
- [あなたの名前]の部分は実際の名前に置き換えること

期待される出力例

```
こんにちは、C言語の世界へ。  
私の名前は 山田太郎 です。  
今日からプログラミングを始めます。
```

ファイル名: ex2_1_hello_intro.c

問題2-2:変数と基本データ型

以下の変数を定義し、値を代入して出力するプログラムを作成してください。

変数一覧

- 整数型: 年齢(例:25)
- 浮動小数点型: 身長(例:170.5)
- 文字型: 血液型(例: 'A')
- 整数型: 好きな数字(例:7)

出力例

```
=== 自己紹介データ ===  
年齢: 25歳  
身長: 170.5cm  
血液型: A型  
好きな数字: 7  
=====
```

要求事項

- 適切なデータ型を選択すること
- printfの書式指定子を正しく使用すること
- コメントで各変数の説明を記述すること

期待される出力例

```
=== 自己紹介データ ===  
年齢: 25歳  
身長: 170.5cm  
血液型: A型  
好きな数字: 7  
=====
```

ファイル名: ex2_2_personal_data.c

問題2-3:入力と出力

ユーザーから名前と年齢を入力してもらい、それを使って計算結果を表示するプログラムを作成してください。

機能

- 1 名前を入力してもらう
- 2 年齢を入力してもらう
- 3 10年後の年齢を計算する
- 4 結果を表示する

出力例

```
お名前を入力してください: 田中太郎
年齢を入力してください: 25

こんにちは、田中太郎さん。
現在の年齢: 25歳
10年後の年齢: 35歳
```

要求事項

- scanf関数を使用すること
- 適切なバッファサイズを設定すること
- 計算結果を変数に保存すること

期待される出力例

```
お名前を入力してください: 田中太郎
年齢を入力してください: 25

こんにちは、田中太郎さん。
現在の年齢: 25歳
10年後の年齢: 35歳
```

ファイル名: ex2_3_age_calculator.c

応用問題

問題2-4:書式指定子の練習

様々な書式指定子を使って、数値を異なる形式で表示するプログラムを作成してください。

表示する数値: 123, 3.14159, 255

出力例

```
=== 書式指定子のデモ ===
整数 123 の表示:
10進数: 123
16進数: 7b
8進数: 173
フィールド幅5: | 123|
ゼロ埋め: |00123|

実数 3.14159 の表示:
デフォルト: 3.14159
小数点以下2桁: 3.14
指数表記: 3.14159e+00
フィールド幅10.2: |      3.14|

文字コード 255:
文字として: y
16進数: ff
10進数: 255
=====
```

要求事項

- 各種書式指定子を使用すること(%d, %x, %o, %f, %e, %cなど)
- フィールド幅とゼロ埋めを実演すること
- 適切なコメントを記述すること

期待される出力例

```
=== 書式指定子のデモ ===
整数 123 の表示:
10進数: 123
16進数: 7b
8進数: 173
フィールド幅5: | 123|
ゼロ埋め: |00123|

実数 3.14159 の表示:
デフォルト: 3.141590
小数点以下2桁: 3.14
指数表記: 3.141590e+000
フィールド幅10.2: |      3.14|

文字コード 65:
文字として: A
16進数: 41
10進数: 65
=====
```

ファイル名: ex2_4_format_demo.c

問題2-5:簡単な計算機

四則演算を実行する簡単な計算機プログラムを作成してください。

機能

- 1 2つの数値を入力してもらう
- 2 四則演算(+, -, *, /)の結果を表示する
- 3 割り算では整数除算と実数除算の両方を表示する

出力例

```
簡単な計算機プログラム
=====
第1の数値を入力してください: 7
第2の数値を入力してください: 2

計算結果:
7 + 2 = 9
7 - 2 = 5
7 * 2 = 14
7 / 2 = 3 (整数除算)
7 / 2 = 3.50 (実数除算)
```

要求事項

- 整数除算と実数除算を区別すること
- ゼロ除算のチェックは不要(基礎課題のため)
- 適切な変数名を使用すること

期待される出力例

```
簡単な計算機プログラム
=====
第1の数値を入力してください: 10
第2の数値を入力してください: 3

計算結果:
10 + 3 = 13
10 - 3 = 7
10 * 3 = 30
10 / 3 = 3 (整数除算)
10 / 3 = 3.33 (実数除算)
```

ファイル名: ex2_5_simple_calculator.c

チャレンジ問題

問題2-6:文字とASCIIコード

文字とASCIIコードの関係を学ぶプログラムを作成してください。

機能

- 1 ユーザーから文字を入力してもらう
- 2 その文字のASCIIコードを表示する
- 3 ASCIIコード表の一部を表示する

出力例

```
文字を1つ入力してください: A
入力された文字: A
ASCIIコード: 65

ASCII表 (32-126) の一部:
32:  33: ! 34: " 35: # 36: $ 37: % 38: & 39: '
40: ( 41: ) 42: * 43: + 44: , 45: - 46: . 47: /
48: 0 49: 1 50: 2 51: 3 52: 4 53: 5 54: 6 55: 7
56: 8 57: 9 58: : 59: ; 60: < 61: = 62: > 63: ?
64: @ 65: A 66: B 67: C 68: D 69: E 70: F 71: G
```

要求事項

- getchar()またはscanf("%c", &ch)を使用すること
- ループを使ってASCII表を表示すること
- 適切な書式で表を整列させること

期待される出力例

```
文字を1つ入力してください: A
入力された文字: A
ASCIIコード: 65

ASCII表 (32-126) の一部:
32:  33: ! 34: " 35: # 36: $ 37: % 38: & 39: '
40: ( 41: ) 42: * 43: + 44: , 45: - 46: . 47: /
48: 0 49: 1 50: 2 51: 3 52: 4 53: 5 54: 6 55: 7
56: 8 57: 9 58: : 59: ; 60: < 61: = 62: > 63: ?
64: @ 65: A 66: B 67: C 68: D 69: E 70: F 71: G
```

ファイル名: ex2_6_ascii_explorer.c

問題2-7:文字列の基本操作

文字配列を使って基本的な文字列操作を学ぶプログラムを作成してください。

機能

- 1 ユーザーから短い文字列(単語)を入力してもらう

- 2 文字列の各文字を1文字ずつ表示
- 3 文字列を逆順に表示
- 4 各文字のASCIIコードを表示

出力例

単語を入力してください: Hello

入力された文字列: Hello

1文字ずつの表示:

H - e - l - l - o

逆順表示: olleH

各文字のASCIIコード:

H: 72

e: 101

l: 108

l: 108

o: 111

要求事項

- 文字配列を使用すること
- ループを使って文字を1つずつ処理すること
- 文字列の長さを数える処理を含めること

期待される出力例

単語を入力してください: Hello

入力された文字列: Hello

1文字ずつの表示:

H - e - l - l - o

逆順表示: olleH

各文字のASCIIコード:

H: 72

e: 101

l: 108

l: 108

o: 111

ファイル名: ex2_7_string_basics.c

問題2-8:プログラマー情報カード

プログラマーの情報を整形して表示するプログラムを作成してください。

機能

- 1 複数の情報を入力(名前、年齢、経験年数、好きな言語)
- 2 情報をカード形式で綺麗に表示
- 3 経験レベルを判定(初心者/中級者/上級者)

出力例

```
=== プログラマー情報入力 ===
名前: 山田太郎
年齢: 25
プログラミング経験年数: 3
好きなプログラミング言語: C
```

```
+-----+
|   プログラマー情報カード   |
+-----+
| 名前: 山田太郎             |
| 年齢: 25歳                 |
| 経験: 3年                  |
| レベル: 中級者             |
| 好きな言語: C              |
+-----+
```

経験レベルの判定基準

- 0-1年: 初心者
- 2-4年: 中級者
- 5年以上: 上級者

要求事項

- 適切な変数型を使用すること
- 条件分岐で経験レベルを判定すること
- 整形された出力を作成すること

期待される出力例

```
=== プログラマー情報入力 ===
名前: 山田太郎
年齢: 25
プログラミング経験年数: 3
好きなプログラミング言語: C
```

```
+-----+
|   プログラマー情報カード   |
+-----+
| 名前: 山田太郎             |
| 年齢: 25歳                 |
| 経験: 3年                  |
| レベル: 中級者             |
| 好きな言語: C              |
+-----+
```

ファイル名: ex2_8_programmer_card.c

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- 警告を有効にしてコンパイルし、実行結果を確認してください。

コンパイル例

```
# コンパイル例

gcc -o hello_intro ex2_1_hello_intro.c
gcc -o personal_data ex2_2_personal_data.c
gcc -o age_calculator ex2_3_age_calculator.c

# 実行例

./hello_intro
./personal_data
./age_calculator
```

評価基準

- 基本文法:C言語の構文に従っていること
- 可読性:インデント、変数名、コメントが処理内容を表していること
- 機能:要求された入出力と計算を実装していること
- 書式指定:データ型に対応する書式指定子を使用していること

学習項目

- C言語の基本的な入出力方法
- 変数の宣言と初期化
- 基本データ型の理解
- 書式指定子の使い方
- コメントの書き方
- プログラムの基本構造

第3章:データ型と変数の演習問題

基礎問題

演習3-1:変数宣言と初期化

さまざまなデータ型の変数を宣言し、初期化して値を表示するプログラムを作成してください。

要件

- 各基本データ型(char, short, int, long, float, double)の変数を宣言
- 適切な値で初期化
- printf関数で型に応じた書式指定子を使用して出力
- 符号付き・符号なしの両方を含める

期待される出力例

```
char型: 文字 = 'A', 値 = 65
unsigned char型: 値 = 255
short型: 値 = -1000
unsigned short型: 値 = 65535
int型: 値 = -123456
unsigned int型: 値 = 4294967295
float型: 値 = 3.14159
double型: 値 = 3.141592653589793
```

演習3-2:四則演算計算機

2つの数値を変数に格納し、四則演算(加算、減算、乗算、除算)の結果を表示するプログラムを作成してください。

要件

- 整数型と浮動小数点型の両方で実装
- 除算では整数除算と実数除算の違いを表示
- 各演算結果を見やすく表示

期待される出力例

```
=== 整数演算 ===
a = 15, b = 4
加算: 15 + 4 = 19
減算: 15 - 4 = 11
乗算: 15 * 4 = 60
除算: 15 / 4 = 3 (整数除算)

=== 実数演算 ===
x = 15.0, y = 4.0
加算: 15.00 + 4.00 = 19.00
減算: 15.00 - 4.00 = 11.00
乗算: 15.00 * 4.00 = 60.00
除算: 15.00 / 4.00 = 3.75
```

演習3-3:データ型サイズの確認

sizeof演算子を使って、各データ型のサイズを表示するプログラムを作成してください。

要件

- 基本データ型すべてのサイズを表示
- 配列のサイズも確認
- ポインタのサイズも確認
- サイズをバイト単位で表示

期待される出力例

```
=== データ型のサイズ ===
char: 1 バイト
short: 2 バイト
int: 4 バイト
long: 8 バイト
float: 4 バイト
double: 8 バイト

=== 配列のサイズ ===
int array[10]: 40 バイト
要素数: 10

=== ポインタのサイズ ===
char*: 8 バイト
int*: 8 バイト
double*: 8 バイト
```

応用問題

演習3-4:型変換の理解

整数除算と実数除算の違い、および暗黙的・明示的型変換を確認するプログラムを作成してください。

要件

- 同じ数値で整数除算と実数除算を実行し、結果を比較
- 暗黙的型変換が発生する例を実装
- 明示的型変換(キャスト)を使用した例を実装
- 精度の損失が発生する例を示す

期待される出力例

```
=== 整数除算と実数除算 ===
int a = 10, b = 3
整数除算: 10 / 3 = 3
実数除算 (キャスト使用) : 10 / 3 = 3.333333

=== 暗黙的型変換 ===
int i = 10
double d = 3.14
i + d = 13.140000 (intがdoubleに変換)

=== 明示的型変換 (キャスト) ===
double x = 3.75
(int)x = 3 (小数部分が切り捨て)

=== 精度の損失 ===
float f = 1234567890.123456
表示: 1234567936.000000 (精度が失われる)
```

演習3-5:スコープの実験

グローバル変数、ローカル変数、静的変数を使い分けるプログラムを作成してください。

要件

- 同じ名前の変数を異なるスコープで宣言
- スコープの隠蔽(シャドウイング)を実演
- 静的変数を使ったカウンター関数を実装
- 各変数の値の変化を追跡して表示

期待される出力例

```
=== スコープの実験 ===
グローバル変数 x = 100

main関数内:
ローカル変数 x = 10 (グローバル変数を隠蔽)

ブロック内:
ブロック内の x = 5 (main関数の変数を隠蔽)
ブロックを出た後の x = 10

=== 静的変数カウンター ===
counter()の呼び出し 1回目: 1
counter()の呼び出し 2回目: 2
counter()の呼び出し 3回目: 3

グローバル変数 x = 100 (変更されていない)
```

演習3-6:定数の活用

constと#defineを使って定数を定義し、円の面積と円周を計算するプログラムを作成してください。

要件

- 円周率をconstと#defineの両方で定義
- 半径を入力として受け取る
- 面積と円周を計算して表示
- 定数を変更しようとした場合のエラーを確認(コメントで説明)

期待される出力例

```
円の計算プログラム
=====
半径を入力してください: 5.0

計算結果:
半径: 5.00
円周: 31.42 (#define版の円周率使用)
面積: 78.54 (const版の円周率使用)

両方の円周率の値:
#define PI = 3.141593
const pi = 3.141593
```

チャレンジ問題

演習3-7:温度変換プログラム

摂氏・華氏・ケルビンの温度単位を相互変換するプログラムを作成してください。

要件

- 変換式で使用する定数を適切に定義
- 浮動小数点演算の精度に注意
- ユーザーフレンドリーな入出力
- 変換公式。
 - 華氏 = 摂氏 × 9/5 + 32
 - ケルビン = 摂氏 + 273.15

期待される出力例

```
=== 温度変換プログラム ===
温度を入力してください (摂氏) : 25.0

変換結果:
摂氏: 25.00 °C
華氏: 77.00 °F
ケルビン: 298.15 K

別の例 (氷点) :
摂氏: 0.00 °C
華氏: 32.00 °F
ケルビン: 273.15 K

別の例 (沸点) :
摂氏: 100.00 °C
華氏: 212.00 °F
ケルビン: 373.15 K
```

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ファイルの冒頭に問題番号と扱うデータ型をコメントで記載してください。
- C90規格と警告オプションを指定してコンパイルしてください。

コンパイル例

```
gcc -std=c90 -Wall -Wextra -pedantic ex_variables.c -o ex_variables
./ex_variables
```

ヒント

- printf関数の書式指定子を正しく使用する
 - %d: int
 - %u: unsigned int
 - %ld: long
 - %f: float/double
 - %c: char
- 初期化されていない変数の使用に注意
- 型の範囲を超えないよう注意
- sizeof演算子の戻り値はsize_t型(%luで出力)

第4章:演算子の演習問題

基礎問題

問題4-1:四則演算計算機

2つの整数を入力として受け取り、すべての算術演算(加算、減算、乗算、除算、剰余)の結果を表示するプログラムを作成してください。

要件

- ユーザーから2つの整数を入力
- 各演算の結果を見やすく表示
- ゼロ除算のチェックを含める

期待される出力例

```
2つの整数を入力してください: 10 3
10 + 3 = 13
10 - 3 = 7
10 * 3 = 30
10 / 3 = 3
10 % 3 = 1
```

ファイル名: ex4_1_calculator.c

問題4-2:比較と論理演算

2つの整数を入力として受け取り、関係演算子と論理演算子を使って最大値と最小値を求めるプログラムを作成してください。

要件

- 2つの整数を入力
- if文を使わず、条件演算子(三項演算子)のみで実装
- 最大値と最小値を表示

期待される出力例

```
2つの整数を入力してください: 15 8
最大値: 15
最小値: 8

別の例:
2つの整数を入力してください: -5 3
最大値: 3
最小値: -5
```

ファイル名: ex4_2_comparison.c

問題4-3:インクリメント・デクリメント

前置と後置のインクリメント・デクリメント演算子の違いを確認するプログラムを作成してください。

要件

- 変数に対して前置・後置の両方を使用
- 各操作後の値を表示
- 式の中での動作も確認

期待される出力例

```
=== インクリメント・デクリメント演算子の動作確認 ===
初期値: x = 5

後置インクリメント (x++): 5
操作後の x: 6

前置インクリメント (++x): 7
操作後の x: 7

初期値: y = 10
式の中での動作:
a = y++ + 2 = 12 (y = 11)
b = ++y + 2 = 14 (y = 12)

デクリメントも同様:
初期値: z = 8
c = z-- = 8 (z = 7)
d = --z = 6 (z = 6)
```

ファイル名: ex4_3_increment.c

応用問題

問題4-4:ビット操作

整数を入力として受け取り、その数値の各ビットを表示し、特定のビット操作を実行するプログラムを作成してください。

要件

- 2進数表示
- 指定したビット位置のON/OFF確認
- 特定ビットの設定・クリア・トグル機能

期待される出力例

```
整数を入力してください: 10
10 の2進数表示: 00001010

ビット位置を入力してください (0-7) : 2
ビット位置 2 は OFF です

操作を選択してください:

1. ビットを設定 (ON)
2. ビットをクリア (OFF)
3. ビットをトグル (反転)

選択: 1

操作後: 14 (00001110)
ビット位置 2 を ON にしました
```

ファイル名: ex4_4_bitwise.c

問題4-5:条件演算子の活用

3つの数値を入力として受け取り、条件演算子(三項演算子)のみを使って昇順に並び替えるプログラムを作成してください。

要件

- if文、switch文を使用しない
- 条件演算子のネストを活用
- 並び替えた結果を表示

期待される出力例

3つの整数を入力してください: 9 3 6
昇順に並び替え: 3 6 9

別の例:
3つの整数を入力してください: -5 10 0
昇順に並び替え: -5 0 10

ファイル名: ex4_5_ternary_sort.c

問題4-6: 演算子優先順位

複雑な式を含むプログラムを作成し、演算子の優先順位による計算結果の違いを確認してください。

要件

- 同じ数値で異なる式を評価
- 括弧の有無による結果の違いを表示
- 少なくとも3種類以上の演算子を使用

期待される出力例

```
=== 演算子優先順位の確認 ===  
a = 2, b = 3, c = 4  
  
式1: a + b * c = 14  
説明: 乗算が先 □ 2 + (3 * 4) = 2 + 12 = 14  
  
式2: (a + b) * c = 20  
説明: 括弧内が先 □ (2 + 3) * 4 = 5 * 4 = 20  
  
式3: a << b + c = 256  
説明: 加算が先 □ 2 << (3 + 4) = 2 << 7 = 256  
  
式4: a & b | c = 6  
説明: &が先、次に| □ (2 & 3) | 4 = 2 | 4 = 6
```

ファイル名: ex4_6_precedence.c

チャレンジ問題

問題4-7: 複合代入演算子の応用

複合代入演算子をすべて使い、数値を変換するプログラムを作成してください。対象は+=、-=、*=、/=、%=、&=、|=、^=、<<=、>>=です。

要件

- 初期値から各種複合代入演算子を使って変換
- 各ステップでの値の変化を表示
- ビット演算の複合代入も含める

期待される処理

- 1 初期値を入力
- 2 各種演算を順番に適用
- 3 最終結果と各ステップを表示

ファイル名: ex4_7_compound_operators.c

問題4-8: ビット演算による最適化

ビット演算を使って、以下の処理を高速に実行するプログラムを作成してください。

実装する機能

- 1 2の累乗かどうかの判定
- 2 2の累乗による乗算・除算
- 3 偶数・奇数の判定
- 4 数値の符号反転(2の補数)
- 5 最下位の1ビットを抽出

要件

- 通常の演算子を使わずビット演算のみで実装
- 各機能の動作原理をコメントで説明
- テストケースを含める

ファイル名: ex4_8_bit_optimization.c

問題4-9:演算子組み合わせパズル

与えられた数値と演算子を使って、目標の値を作るプログラムを作成してください。

仕様

- 1 4つの1桁の数字を入力
- 2 目標値を入力
- 3 +, -, *, / の演算子を使用
- 4 括弧の使用も可能
- 5 すべての数字を1回ずつ使用
- 6 解が存在する場合は式を表示

例

```
数字を4つ入力: 2 3 4 5
目標値: 24
解: (2 + 3) * 4 + 5 - 5 = 24
```

ファイル名: ex4_9_operator_puzzle.c

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ファイルの冒頭に問題番号と使用する演算子をコメントで記載してください。
- C90規格と警告オプションを指定してコンパイルしてください。

コンパイル例

```
gcc -std=c90 -Wall -Wextra -pedantic ex4_1_calculator.c -o ex4_1_calculator
./ex4_1_calculator
```

評価ポイント

- **正確性:** 要求された機能が正しく動作するか
- **エラー処理:** ゼロ除算などの例外的な状況への対処
- **コード品質:** 可読性、適切なコメント、変数名

- 効率性:特にビット演算問題での最適化

ヒント

- 演算子の優先順位に注意
- ゼロ除算やオーバーフローに対する考慮
- 可読性を意識したコードを心がける
- 適切なコメントを追加する
- ビット演算では2進数表現を意識する

第5章:条件分岐の演習問題

演習の目的

- if文、switch文の理解を深める
- 条件演算子の適切な使い方を習得する
- 複雑な条件分岐の設計力を養う

基礎問題

問題5-1:年齢による料金計算

映画館の料金システムを実装してください。

要件

- 3歳未満: 無料
- 3歳以上12歳以下: 子供料金(800円)
- 13歳以上18歳以下: 学生料金(1200円)
- 19歳以上65歳以下: 大人料金(1800円)
- 66歳以上: シニア料金(1200円)

実装のヒント

- if-else if文を使用
- 境界値に注意

期待される出力例

```
映画館料金計算システム
年齢を入力してください: 25
お客様の料金は 1800円 です (大人料金)

別の例:
年齢を入力してください: 5
お客様の料金は 800円 です (子供料金)

年齢を入力してください: 70
お客様の料金は 1200円 です (シニア料金)
```

ファイル名: ex5_1_movie_price.c

問題5-2:電卓プログラム

四則演算ができる簡単な電卓を作成してください。

要件

- ユーザーから2つの数値と演算子(+, -, *, /)を入力
- switch文を使って演算を実行
- ゼロ除算のエラー処理を含める

実装のヒント

- 演算子はchar型で受け取る
- 除算の場合は分母が0でないかチェック

期待される出力例

```
簡単な電卓プログラム
第1の数値を入力: 15
演算子を入力 (+, -, *, /): *
第2の数値を入力: 3
計算結果: 15.00 * 3.00 = 45.00

別の例 (ゼロ除算):
第1の数値を入力: 10
演算子を入力 (+, -, *, /): /
第2の数値を入力: 0
エラー: ゼロで除算することはできません
```

ファイル名: ex5_2_calculator.c

問題5-3: BMI判定プログラム

身長と体重からBMIを計算し、判定するプログラムを作成してください。

要件

- BMI = 体重(kg) / (身長(m) × 身長(m))
- BMI判定基準:
 - 18.5未満: 低体重
 - 18.5以上25未満: 標準体重
 - 25以上30未満: 肥満度1
 - 30以上: 肥満度2

実装のヒント

- 身長はセンチメートルで入力してメートルに変換
- 浮動小数点数を使用

期待される出力例

```
BMI判定プログラム
身長(cm)を入力してください: 170
体重(kg)を入力してください: 65

BMI値: 22.49
判定: 標準体重です

別の例:
身長(cm)を入力してください: 165
体重(kg)を入力してください: 80

BMI値: 29.38
判定: 肥満度1です
```

ファイル名: ex5_3_bmi_check.c

応用問題

問題5-4: うるう年判定

西暦年を入力してうるう年かどうか判定するプログラムを作成してください。

うるう年の条件

- 1 4で割り切れる年はうるう年
- 2 ただし、100で割り切れる年は平年
- 3 ただし、400で割り切れる年はうるう年

実装のヒント

- 複数の条件を論理演算子で組み合わせる
- 条件の順序に注意

期待される出力例

```
うるう年判定プログラム
西暦年を入力してください: 2024
2024年はうるう年です

別の例:
西暦年を入力してください: 1900
1900年は平年です

西暦年を入力してください: 2000
2000年はうるう年です
```

ファイル名: ex5_4_leap_year.c

問題5-5:成績評価システム

複数の科目の点数から総合評価を出すプログラムを作成してください。

要件

- 3科目(国語、数学、英語)の点数を入力
- 平均点を計算
- 平均点による評価(A~F)
- 全科目60点以上の場合のみ「合格」、それ以外は「不合格」
- 条件演算子を使って簡潔に表示

評価基準

- 90点以上: A
- 80点以上90点未満: B
- 70点以上80点未満: C
- 60点以上70点未満: D
- 50点以上60点未満: E
- 50点未満: F

実装のヒント

- 平均点の計算は整数演算に注意
- 複数の条件を組み合わせる

期待される出力例

```
成績評価システム
国語の点数: 85
数学の点数: 92
英語の点数: 78
```

```
平均点: 85.00
評価: B
判定: 合格
```

```
別の例 (不合格):
国語の点数: 75
数学の点数: 55
英語の点数: 80
```

```
平均点: 70.00
評価: C
判定: 不合格 (数学が60点未満)
```

ファイル名: ex5_5_grade_system.c

問題5-6: 曜日判定プログラム

年月日を入力して、その日の曜日を判定するプログラムを作成してください。

要件

- ツェラーの公式を使用して曜日を計算
- 入力値の妥当性チェック(月は1-12、日は1-31など)
- 曜日を日本語で表示

ツェラーの公式(簡易版)

- 1月、2月は前年の13月、14月として計算

期待される出力例

```
曜日判定プログラム
年を入力: 2024
月を入力: 3
日を入力: 15
```

```
2024年3月15日は金曜日です
```

```
別の例:
年を入力: 2023
月を入力: 12
日を入力: 25
```

```
2023年12月25日は月曜日です
```

ファイル名: ex5_6_weekday.c

チャレンジ問題

問題5-7: じゃんけんゲーム

コンピューターとじゃんけんをするプログラムを作成してください。

要件

- ユーザーの手を数値で入力(1:グー、2:チョキ、3:パー)
- コンピューターの手はランダムに決定
- 勝敗を判定して表示
- 不正な入力のチェック

追加要件

- 3回勝負で最終的な勝者を決定
- 各回の結果を記録して最後に表示
- 連勝ボーナスポイントシステム

期待される出力例

```
=== じゃんけんゲーム（3回勝負） ===

第1回戦
あなたの手を選んでください（1:グー、2:チョキ、3:パー）： 1
あなた：グー
コンピューター：チョキ
結果：あなたの勝ち。

第2回戦
あなたの手を選んでください（1:グー、2:チョキ、3:パー）： 2
あなた：チョキ
コンピューター：チョキ
結果：あいこ

第3回戦
あなたの手を選んでください（1:グー、2:チョキ、3:パー）： 3
あなた：パー
コンピューター：グー
結果：あなたの勝ち。（連勝ボーナス +1)

=== 最終結果 ===
あなた：3ポイント（2勝0敗1分）
コンピューター：0ポイント
優勝：あなた。
```

ファイル名：ex5_7_janken_game.c

問題5-8:交通信号シミュレーター

信号機の動作をシミュレートするプログラムを作成してください。

要件

- 現在の信号の色と経過時間から次の動作を決定
- 歩行者用信号も含める
- 緊急車両通過モード(すべて赤信号)
- 時間帯による点滅信号への切り替え

信号のパターン

- 青(30秒)→黄(3秒)→赤(30秒)
- 歩行者：青(20秒)→点滅(10秒)→赤(33秒)

期待される出力例

```
交通信号シミュレーター
現在の信号状態を入力（1:青、2:黄、3:赤）： 1
経過時間（秒）： 28
現在時刻（0-23時）： 14

=== 信号状態 ===
車両用信号：青（残り2秒）
歩行者信号：赤
次の状態：まもなく黄信号に変わります

別の例（深夜）：
現在時刻（0-23時）： 2
深夜モード：黄色点滅信号

別の例（緊急車両）：
緊急車両通過モード（1:有効、0:無効）： 1
すべての信号：赤（緊急車両通過中）
```

ファイル名: ex5_8_traffic_light.c

問題5-9:複雑な料金計算システム

タクシー料金計算システムを実装してください。

要件

- ・ 初乗り料金: 1.052kmまで420円
- ・ 加算料金: 以降237mごとに80円
- ・ 時間距離併用: 時速10km以下では90秒ごとに80円
- ・ 深夜割増(22:00-5:00): 2割増
- ・ 予約料金、高速料金の加算オプション

入力項目

- ・ 走行距離、走行時間、時刻、オプション選択

期待される出力例

```
タクシー料金計算システム
走行距離(km): 5.5
走行時間(分): 25
現在時刻(0-23): 23
予約利用(1:あり、0:なし): 0
高速道路利用(1:あり、0:なし): 0

=== 料金明細 ===
初乗り料金: 420円
加算料金: 760円 (4.448km分)
深夜割増 (20%): 236円
-----
合計料金: 1,416円

別の例 (渋滞時):
走行距離(km): 2.0
走行時間(分): 30
現在時刻(0-23): 18

=== 料金明細 ===
初乗り料金: 420円
加算料金: 320円 (距離加算)
時間加算料金: 640円 (低速走行)
-----
合計料金: 1,380円
```

ファイル名: ex5_9_taxi_fare.c

提出物

- ・ 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ・ ファイルの冒頭に問題番号と判定条件をコメントで記載してください。
- ・ 警告を有効にしてコンパイルし、境界値を含む実行結果を確認してください。

評価基準

- ・ 正確性: 要求仕様を満たしているか
- ・ 可読性: 適切なインデント、変数名、コメント
- ・ 効率性: 不要な条件分岐がないか
- ・ エラー処理: 不正な入力への対処

学習のポイント

- 条件の順序と網羅性
- switch文とif文の使い分け
- 条件演算子による簡潔な記述
- 複雑な条件の論理的な整理

第6章:繰り返し処理の演習問題

演習の目的

- for文、while文、do-while文の使い分けを理解する
- break文とcontinue文を適切に使えるようになる
- ネストしたループを活用した問題解決能力を養う

基礎問題

問題6-1:フィボナッチ数列

最初のN個のフィボナッチ数を表示するプログラムを作成してください。

要件

- ユーザーから表示する個数Nを入力
- フィボナッチ数列: 0, 1, 1, 2, 3, 5, 8, 13, ...
- for文を使用して実装

実装のヒント

- 最初の2つの数は0と1
- 3番目以降は前の2つの数の和

期待される出力例

```
フィボナッチ数列表示プログラム
表示する個数を入力してください: 10

フィボナッチ数列（最初の10個）:
0 1 1 2 3 5 8 13 21 34

別の例:
表示する個数を入力してください: 15

フィボナッチ数列（最初の15個）:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

ファイル名: ex6_1_fibonacci.c

問題6-2:数当てゲーム

1から100までの数当てゲームを作成してください。

要件

- プログラムがランダムに1～100の数を決定
- ユーザーが予想を入力
- 「もっと大きい」「もっと小さい」のヒントを出す
- 10回以内に当てられなければゲームオーバー
- do-while文を使用

実装のヒント

- `rand() % 100 + 1` でランダムな数を生成
- 試行回数をカウント

期待される出力例

```
=== 数当てゲーム ===
1から100までの数を当ててください。
10回以内に当てられればクリアです。

あなたの予想: 50
もっと小さい数です

あなたの予想: 25
もっと大きい数です

あなたの予想: 37
もっと大きい数です

あなたの予想: 43
もっと小さい数です

あなたの予想: 40
正解です。
4回で当てました。

別の例（失敗）：
（10回予想しても当たらない場合）
残念。答えは 72 でした。
```

ファイル名: ex6_2_number_game.c

問題6-3:掛け算表

9×9の掛け算表を表示するプログラムを作成してください。

要件

- ネストしたfor文を使用
- 見やすい表形式で出力
- 列と行のヘッダーを含める

出力例

```
  1  2  3  4  5  6  7  8  9
1  1  2  3  4  5  6  7  8  9
2  2  4  6  8 10 12 14 16 18
3  3  6  9 12 15 18 21 24 27
...
```

実装のヒント

- printf()の書式指定子で桁揃えを行う
- %3dなどを使用して桁数を統一

ファイル名: ex6_3_multiplication_table.c

応用問題

問題6-4:素数リスト

指定された範囲内のすべての素数を表示するプログラムを作成してください。

要件

- ユーザーから範囲(開始値と終了値)を入力
- その範囲内の素数をすべて表示
- 素数の個数も表示
- ネストしたループとbreak文を活用

実装のヒント

- 2から $\sqrt{n}$ まで割り切れるか確認
- 効率化のため、偶数は2以外スキップ

期待される出力例

```
素数リスト表示プログラム
開始値を入力してください: 10
終了値を入力してください: 50

10から50までの素数:
11 13 17 19 23 29 31 37 41 43 47

素数の個数: 11個

別の例:
開始値を入力してください: 1
終了値を入力してください: 30

1から30までの素数:
2 3 5 7 11 13 17 19 23 29

素数の個数: 10個
```

ファイル名: ex6_4_prime_list.c

問題6-5:図形の描画

アスタリスク(*)を使って様々な図形を描画するプログラムを作成してください。

要件

以下の4つの図形を描画する機能を実装:

- 1 正方形
- 2 直角三角形
- 3 逆直角三角形
- 4 ダイヤモンド

例(サイズ5の場合)

正方形:

```
*****
*****
*****
*****
*****
```

直角三角形:

```
*
**
***
****
*****
```

逆直角三角形:

```
*****
****
***
**
*
```

ダイヤモンド:

```
  *
 ***
```

```
*****
***
*
```

実装のヒント

- ネストしたループを使用
- 空白文字とアスタリスクの配置に注意

ファイル名: ex6_5_draw_shapes.c

問題6-6:完全数の探索

完全数を探すプログラムを作成してください。

完全数とは

その数自身を除く約数の和が、その数自身と等しくなる自然数例: $6 = 1 + 2 + 3$, $28 = 1 + 2 + 4 + 7 + 14$

要件

- ユーザーから上限値を入力
- 1からその値までの完全数をすべて表示
- 各完全数について、約数も表示

期待される出力例

```
完全数探索プログラム
上限値を入力してください: 1000

1から1000までの完全数:

6は完全数です
約数: 1 + 2 + 3 = 6

28は完全数です
約数: 1 + 2 + 4 + 7 + 14 = 28

496は完全数です
約数: 1 + 2 + 4 + 8 + 16 + 31 + 62 + 124 + 248 = 496

見つかった完全数の個数: 3個
```

ファイル名: ex6_6_perfect_numbers.c

チャレンジ問題

問題6-7:パスカルの三角形

パスカルの三角形を表示するプログラムを作成してください。

要件

- ユーザーから行数を入力(最大15行)
- 適切な配置で三角形を表示
- 各数値は前の行の2つの数の和

出力例(5行の場合)

```
  1
 1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

実装のヒント

- 2次元配列を使用
- 適切な空白で中央揃え

ファイル名: ex6_7_pascal_triangle.c

問題6-8:コンソールアニメーション

簡単なコンソールアニメーションを作成してください。

要件

- 横に移動するアスタリスクのアニメーション
- 画面幅を考慮して左右に往復
- ESCキーまたはCtrl+Cで終了
- system(“clear”)またはsystem(“cls”)を使用

動作例

```
*                (右に移動)

*
 *
  *

   *                (端に到達したら折り返し)

    *
   *
  *
 *

*                (左端に戻る)
```

期待される出力例

```
コンソールアニメーション
Ctrl+Cで終了します...

(画面がクリアされて以下のアニメーションが繰り返される)

フレーム1:
*

フレーム2:
 *

フレーム3:
  *

... (続く) ...

フレーム40:
                        *

(端に到達後、逆方向に移動)

フレーム41:
                        *

... (左端まで戻って繰り返す) ...
```

ファイル名: ex6_8_console_animation.c

問題6-9:数字ピラミッド

数字を使ったピラミッドパターンを生成するプログラムを作成してください。

要件

- ユーザーから高さを入力(1-9)

- 複数のパターンを選択可能
- continue文を活用した効率的な実装

パターン例(高さ5)

パターン1:

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

パターン2:

```
1
1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
```

パターン3:

```
1
2 3 2
3 4 5 4 3
4 5 6 7 6 5 4
5 6 7 8 9 8 7 6 5
```

ファイル名: ex6_9_number_pyramid.c

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ファイルの冒頭に問題番号と終了条件をコメントで記載してください。
- 警告を有効にしてコンパイルし、境界値を含む実行結果を確認してください。

評価基準

- **正確性:** 要求仕様を満たしているか
- **効率性:** 適切なループの選択と最適化
- **可読性:** 適切なインデント、変数名、コメント
- **創造性:** チャレンジ問題での工夫

学習のポイント

- for、while、do-whileの使い分け
- breakとcontinueの適切な使用
- ネストループの理解と活用
- ループの効率化技術

第7章:配列の演習問題

演習の目的

- 1次元配列と多次元配列の理解を深める
- 配列を使った実践的な問題解決能力を養う
- 基本的な配列操作を身に付ける

基礎問題

問題7-1:配列の基本操作

整数配列に対する各種操作を実行するプログラムを作成してください。

要件

- 5個の整数を入力して配列に格納
- 以下の操作を実装:
 - 配列の全要素を表示
 - 最大値と最小値を見つける
 - 平均値を計算
 - 配列を逆順に並べ替える
 - 指定した値が配列内に存在するか検索

実装のヒント

- 配列のサイズは#defineで定義
- 逆順は別の配列にコピーまたはその場で交換

期待される出力例

```
配列操作プログラム
5個の整数を入力してください: 34 12 67 89 23

配列の内容: 34 12 67 89 23
最大値: 89
最小値: 12
平均値: 45.00

逆順配列: 23 89 67 12 34

検索する値を入力: 67
値 67 は位置 2 に存在します (0から数えて)

検索する値を入力: 50
値 50 は配列内に存在しません
```

ファイル名: ex7_1_array_operations.c

問題7-2:配列の要素シフト

配列の要素を左右にシフトするプログラムを作成してください。

要件

- 10個の整数を配列に格納
- 以下の操作を実装:
 - 全要素を右に1つシフト(最後の要素は先頭へ)

- 全要素を左に1つシフト(最初の要素は最後へ)
- 指定した数だけシフト
- シフト前後の配列を表示

実装のヒント

- 一時変数を使って端の要素を保存
- ループの方向に注意

期待される出力例

```
配列シフトプログラム
初期配列: 1 2 3 4 5 6 7 8 9 10

右に1シフト: 10 1 2 3 4 5 6 7 8 9
左に1シフト: 2 3 4 5 6 7 8 9 10 1

シフト数を入力してください: 3
右に3シフト: 8 9 10 1 2 3 4 5 6 7

シフト数を入力してください: -2
左に2シフト: 3 4 5 6 7 8 9 10 1 2
```

ファイル名: ex7_2_array_shift.c

問題7-3:配列の統計処理

整数配列に対する基本的な統計処理を行うプログラムを作成してください。

要件

- 10個の整数データを配列に格納
- 以下の統計値を計算:
 - 合計値と平均値
 - 最大値と最小値およびその位置
 - 標準偏差(簡易計算でよい)

実装のヒント

- 標準偏差は各要素と平均の差の二乗の平均の平方根

期待される出力例

```
配列統計処理プログラム
10個の整数を入力してください: 78 65 92 83 71 96 54 88 67 79

=== 統計情報 ===
データ: 78 65 92 83 71 96 54 88 67 79
合計値: 773
平均値: 77.30
最大値: 96 (位置: 5)
最小値: 54 (位置: 6)
標準偏差: 12.85
```

ファイル名: ex7_3_array_statistics.c

応用問題

問題7-4:成績管理システム

複数の学生の複数科目の成績を管理するプログラムを作成してください。

要件

- 5人の学生、4科目(国語、数学、英語、理科)の成績を2次元配列で管理

- 各学生の合計点と平均点を計算
- 各科目の平均点を計算
- 最高得点の学生と科目を表示
- 成績表を見やすく表示

実装のヒント

- 2次元配列grades[][] を使用
- 行が学生、列が科目

期待される出力例

成績管理システム

成績表:

学生	国語	数学	英語	理科	合計	平均
学生1	85	92	78	88	343	85.8
学生2	76	88	91	83	338	84.5
学生3	92	76	85	90	343	85.8
学生4	68	95	73	79	315	78.8
学生5	89	84	88	86	347	86.8
科目平均	82.0	87.0	83.0	85.2		

最高得点: 学生4の数学 95点
クラス全体の平均点: 84.3

ファイル名: ex7_4_grade_management.c

問題7-5:行列演算

2次元配列を使って行列演算を実行するプログラムを作成してください。

要件

- 3x3の行列を2つ入力
- 以下の演算を実装:
 - 行列の加算
 - 行列の減算
 - 行列の乗算
 - 転置行列の計算

実装のヒント

- 行列の乗算: $result[i][j] = \sum (a[i][k] * b[k][j])$
- 各演算結果を見やすく表示

期待される出力例

行列演算プログラム

行列A:

```
1 2 3
4 5 6
7 8 9
```

行列B:

```
9 8 7
6 5 4
3 2 1
```

加算結果 (A + B):

```
10 10 10
10 10 10
10 10 10
```

減算結果 (A - B):

```
-8 -6 -4
-2  0  2
 4  6  8
```

乗算結果 (A x B):

```
30 24 18
84 69 54
138 114 90
```

転置行列 (A^T):

```
1 4 7
2 5 8
3 6 9
```

ファイル名: ex7_5_matrix_operations.c

問題7-6: 頻度分布表

テストの点数分布を分析するプログラムを作成してください。

要件

- 30人分のテスト点数(0-100点)を配列に格納
- 10点刻みで頻度分布表を作成
- ヒストグラムを*を使って表示
- 最頻値の範囲を表示

出力例

```
0- 9: ** (2人)
10-19: *** (3人)
20-29: * (1人)
...
90-99: ***** (8人)
```

ファイル名: ex7_6_frequency_distribution.c

チャレンジ問題

問題7-7: ソートアルゴリズムの実装

配列のソートアルゴリズムを複数実装して比較してください。

要件

- 以下のソートアルゴリズムを実装:
 - バブルソート
 - 選択ソート

- ・ 挿入ソート
- ・ 各アルゴリズムの実行時間を測定
- ・ ランダムな配列、ソート済み配列、逆順配列で性能比較

追加要件

- ・ 比較回数と交換回数をカウント
- ・ 結果をグラフィカルに表示(*を使った簡易グラフ)

期待される出力例

```
ソートアルゴリズムの比較

配列サイズ: 20
元の配列: 34 7 23 32 5 62 78 4 87 12 45 28 91 15 53 67 39 22 84 10

=== バブルソート ===
ソート後: 4 5 7 10 12 15 22 23 28 32 34 39 45 53 62 67 78 84 87 91
比較回数: 190
交換回数: 92
実行時間: 0.002ms

=== 選択ソート ===
ソート後: 4 5 7 10 12 15 22 23 28 32 34 39 45 53 62 67 78 84 87 91
比較回数: 190
交換回数: 19
実行時間: 0.001ms

=== 挿入ソート ===
ソート後: 4 5 7 10 12 15 22 23 28 32 34 39 45 53 62 67 78 84 87 91
比較回数: 92
交換回数: 92
実行時間: 0.001ms

効率性比較グラフ (比較回数):
バブル: ***** (190)
選択: ***** (190)
挿入: ***** (92)
```

ファイル名: ex7_7_sort_algorithms.c

問題7-8:ライフゲーム

コンウェイのライフゲームの簡易版を実装してください。

要件

- ・ 20x20の2次元配列でセルの状態を管理
- ・ セルは生(1)または死(0)の状態
- ・ 各世代で以下のルールを適用:
 - ・ 生きているセルの周囲に2-3個の生きたセルがある→生存
 - ・ 死んでいるセルの周囲にちょうど3個の生きたセルがある→誕生
 - ・ それ以外→死亡
- ・ 初期状態をランダムまたはパターンで設定
- ・ 各世代の状態を表示

期待される出力例

コンウェイのライフゲーム (20x20)

世代 0:

```
.....  
.....***.....  
.....*.*.....  
.....*.....  
.....*.....  
.....  
(以下省略)
```

世代 1:

```
.....  
.....*.....  
.....*.*.....  
.....*.....  
.....  
(以下省略)
```

世代 2:

```
.....  
.....***.....  
.....*.*.....  
.....*.....  
.....  
(パターンが繰り返される)
```

10世代後の生存セル数: 8

ファイル名: ex7_8_life_game.c

問題7-9:魔方陣の生成と検証

魔方陣を生成・検証するプログラムを作成してください。

要件

- 3x3の魔方陣を生成(1-9の数字を使用)
- 既存の配列が魔方陣かどうかを検証
- 検証条件:
 - 各行の合計が等しい
 - 各列の合計が等しい
 - 対角線の合計が等しい
 - 1-9の数字が重複なく使用されている
- 複数の魔方陣パターンを生成可能

期待される出力例

```
魔方陣生成・検証プログラム

生成された魔方陣:
2  7  6
9  5  1
4  3  8

検証結果:
行の合計: 15, 15, 15 (PASS)
列の合計: 15, 15, 15 (PASS)
対角線の合計: 15, 15 (PASS)
すべての数字が1-9で重複なし (PASS)

これは正しい魔方陣です。

別の魔方陣パターン:
8  1  6
3  5  7
4  9  2

検証結果: 正しい魔方陣です。
```

ファイル名: ex7_9_magic_square.c

提出物

- ・ 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ・ ファイルの冒頭に問題番号と実装内容をコメントで記載してください。
- ・ 警告を有効にしてコンパイルし、実行結果を確認してください。

評価基準

- ・ **正確性**: 要求仕様を満たしているか
- ・ **効率性**: 配列操作の最適化
- ・ **可読性**: 適切なインデント、変数名、コメント
- ・ **境界処理**: 配列の境界チェック

学習のポイント

- ・ 配列の初期化と要素アクセス
- ・ 多次元配列の使い方
- ・ 配列を使った効率的なデータ処理
- ・ 配列の境界を超えないための注意点

第8章:ポインターの演習問題

基礎問題

演習8-1:ポインターの基本操作

2つの整数変数の値をポインターを使って交換するプログラムを作成してください。

要件

- 2つの整数変数を宣言・初期化
- ポインターを使った値の交換関数を実装
- 交換前後の値とアドレスを表示
- 直接的な値の交換とポインター経由の交換を比較

期待される出力例

```
交換前: a = 10 (アドレス: 0x7fffbffac), b = 20 (アドレス: 0x7fffbffa8)
ポインターを使った交換実行...
交換後: a = 20 (アドレス: 0x7fffbffac), b = 10 (アドレス: 0x7fffbffa8)
```

演習8-2:配列とポインター

ポインター演算を使って配列の要素を逆順に表示するプログラムを作成してください。

要件

- 整数配列を宣言・初期化
- ポインター演算で配列の最後から最初に向かってアクセス
- インデックス記法とポインター記法の両方で表示
- 配列のサイズを動的に計算

期待される出力例

```
配列の内容 (順方向):
インデックス記法: arr[0]=1, arr[1]=2, arr[2]=3, arr[3]=4, arr[4]=5
ポインター記法: *ptr=1, *(ptr+1)=2, *(ptr+2)=3, *(ptr+3)=4, *(ptr+4)=5

配列の内容 (逆順):
ポインター演算で逆順表示: 5 4 3 2 1

各要素のアドレス:
&arr[0] = 0x7fffc000
&arr[1] = 0x7fffc004
&arr[2] = 0x7fffc008
&arr[3] = 0x7fffc00c
&arr[4] = 0x7fffc010
```

演習8-3:文字列操作

ポインターを使って文字列の長さを計算し、文字列を逆順にするプログラムを作成してください。

要件

- 文字列の長さ計算関数(strlen相当)をポインターで実装
- 文字列を逆順にする関数をポインターで実装
- 元の文字列と逆順文字列を表示
- 文字列リテラルと文字配列の違いを考慮

期待される出力例

```
文字列操作プログラム
文字列を入力: Hello World

元の文字列: Hello World
文字列の長さ: 11文字

文字列を逆順に変換中...
逆順文字列: dlroW olleH

各文字とそのアドレス:
'H' at 0x7fffc100
'e' at 0x7fffc101
'l' at 0x7fffc102
... (以下省略)
```

応用問題

演習8-4:配列操作関数

ポインターを使って配列の最大値、最小値、平均値を計算する関数群を作成してください。

要件

- 最大値を見つけてそのポインターを返す関数
- 最小値を見つけてそのポインターを返す関数
- 平均値を計算する関数(戻り値はdouble)
- 各関数の結果を使ってレポートを作成

実装する関数

```
int* find_max(int *arr, int size);
int* find_min(int *arr, int size);
double calculate_average(int *arr, int size);
void print_statistics(int *arr, int size);
```

期待される出力例

```
配列統計分析プログラム
配列: 34 12 67 89 23 45 78 56 90 11

=== 統計情報 ===
最大値: 90 (アドレス: 0x7fffc120, インデックス: 8)
最小値: 11 (アドレス: 0x7fffc124, インデックス: 9)
平均値: 50.50

値の範囲: 11 ~ 90
```

演習8-5:文字列処理

ポインターを使って文字列の検索、置換、分割を実行する関数群を作成してください。

要件

- 文字列内で特定の文字を検索する関数
- 文字列内の文字を置換する関数
- 文字列を特定の文字で分割する関数
- 大文字・小文字を無視した比較機能

期待される出力例

```
文字列処理プログラム
文字列: Hello,World,Programming
```

```
文字 'o' の検索:
見つかった位置: 4, 7, 18
```

```
文字 'o' を '0' に置換:
置換後: Hello,W0rld,Pr0gramming
```

```
カンマで分割:
1: Hello
2: World
3: Programming
```

```
大文字小文字を無視した検索 ('w'):
見つかった位置: 6 (W)
```

演習8-6:データ変換

ポインターを使って配列のデータ型変換を実行するプログラムを作成してください。

要件

- int配列をfloat配列に変換
- 文字列を数値配列に変換
- バイト配列を整数として解釈
- エラー処理の実装

期待される出力例

```
データ型変換プログラム

=== int配列からfloat配列への変換 ===
元のint配列: 10, 20, 30, 40, 50
変換後のfloat配列: 10.00, 20.00, 30.00, 40.00, 50.00

=== 文字列から数値配列への変換 ===
文字列: "12 34 56 78 90"
数値配列: 12, 34, 56, 78, 90

=== バイト配列を整数として解釈 ===
バイト配列: 0x01 0x02 0x03 0x04
リトルエンディアン解釈: 67305985 (0x04030201)
ビッグエンディアン解釈: 16909060 (0x01020304)
```

発展問題

演習8-7:メモリ操作

ポインターを使って任意のデータ型の配列をコピーする汎用関数を作成してください。

要件

- void *ポインターを使った汎用コピー関数
- バイト単位でのメモリコピー
- 型安全性の考慮
- コピー対象の境界チェック

関数プロトタイプ

```
void* generic_copy(void *dest, const void *src, size_t size);
int compare_memory(const void *ptr, const void *ptr, size_t size);
```

演習8-8:アルゴリズム実装

ポインターを使って各種ソートアルゴリズムを実装してください。

要件

- バブルソート(ポインター版)
- 選択ソート(ポインター版)
- 挿入ソート(ポインター版)
- 汎用的な比較関数の使用

演習8-9:データ構造

ポインターを使って簡単なリンクリスト構造を実装してください。

要件

- ノード構造体の定義
- リストへの要素追加・削除
- リストの走査・検索
- メモリリークの防止

構造体例

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;
```

チャレンジ問題

演習8-10:動的配列シミュレータ

malloc/freeを使わずに、静的配列とポインターを使って動的配列のような動作を実現してください。

要件

- 固定サイズの大きな配列をメモリプールとして使用
- 要素の追加・削除・挿入機能
- メモリの断片化管理
- ガベージコレクション機能

提出物

- 問題ごとに独立したCファイルを作成し、処理内容が分かるファイル名を付けてください。
- ファイルの冒頭に問題番号とポインターの用途をコメントで記載してください。
- C90規格と警告オプションを指定してコンパイルしてください。

コンパイル例

```
gcc -std=c90 -Wall -Wextra -pedantic ex_pointer_swap.c -o ex_pointer_swap
./ex_pointer_swap
```

ヒント

ポインターの基本

- 宣言: `int *ptr;`
- アドレス取得: `ptr = &variable;`
- 値の取得: `value = *ptr;`
- 演算: `ptr + 1` は次の要素を指す

安全なプログラミング

- NULLポインタのチェック: `if (ptr != NULL)`
- 配列の境界チェック
- 初期化されていないポインタの使用を避ける

デバッグのコツ

- アドレスと値を分けて表示
- ポインタ演算の結果を段階的に確認
- コンパイラーの警告に注意を払う

よくある間違い

- `*ptr++` と `(*ptr)++` の違い
- ローカル変数のアドレスを返すこと
- 配列の範囲外アクセス
- NULLポインタの参照

第9章:配列とポインタの演習問題

演習の目的

- 配列とポインタの関係を深く理解する
- ポインタ演算を使って配列を操作できるようになる
- 関数への配列の渡し方を習得する
- より効率的な配列処理技術を身につける

基礎問題

問題9-1:ポインタによる配列走査

ポインタを使って配列を操作するプログラムを作成してください。

要件

- 10個の整数を配列に格納
- 以下の操作をポインタを使って実装:
 - 配列の全要素を表示(配列記法を使わない)
 - 最大値と最小値を見つける(ポインタ演算使用)
 - 配列を逆順に並べ替える(ポインタで実装)
 - 特定の値を検索し、そのアドレスを返す

実装のヒント

- `arr[i]`の代わりに`*(arr + i)`を使用
- ポインタのインクリメント/デクリメントを活用

期待される出力例

```
=== ポインタによる配列操作 ===  
配列の内容: 34 67 12 89 23 45 78 56 90 11
```

ポインタ記法での表示:

```
*(ptr+0) = 34  
*(ptr+1) = 67  
*(ptr+2) = 12  
... (続く)
```

最大値: 90 (アドレス: 0x7fffc120)

最小値: 11 (アドレス: 0x7fffc124)

配列を逆順に並べ替え中...

逆順配列: 11 90 56 78 45 23 89 12 67 34

値 45 を検索...

値 45 は アドレス 0x7fffc114 に見つかりました (インデックス: 5)

問題9-2:関数への配列渡し

配列を関数に渡して操作するプログラムを作成してください。

要件

- 配列操作を行う以下の関数を実装:
 - 配列の要素をすべてn倍する関数
 - 2つの配列を要素ごとに加算する関数

- 配列内の最大値へのポインターを返す関数
- 配列を指定位置で分割する関数

実装のヒント

- 関数の引数は `int *arr` または `int arr[]`
- サイズ情報は別引数で渡す

期待される出力例

```
=== 関数への配列渡し ===
元の配列: 1 2 3 4 5

配列の要素を3倍にする:
結果: 3 6 9 12 15

配列A: 10 20 30 40 50
配列B: 5 10 15 20 25
要素ごとの加算結果: 15 30 45 60 75

配列内の最大値: 75 (アドレス: 0x7fff1a4)

配列を位置3で分割:
前半: 15 30 45
後半: 60 75
```

応用問題

問題9-3: 文字列のポインター操作

文字列をポインターで操作するプログラムを作成してください。

要件

- ポインターを使った以下の文字列関数を実装:
 - 文字列の長さを計算(`strlen`相当)
 - 文字列をコピー(`strcpy`相当)
 - 文字列を連結(`strcat`相当)
 - 文字列を反転

実装のヒント

- `char *`を使った操作
- ヌル終端文字の処理に注意

期待される出力例

```
=== 文字列のポインター操作 ===
文字列1: Hello
文字列2: World

my_strlen("Hello") = 5

文字列コピー実行...
コピー結果: Hello

文字列連結実行...
連結結果: HelloWorld

文字列反転実行...
反転結果: dlroWolleH
```

問題9-4: 多次元配列とポインター

2次元配列をポインターで操作するプログラムを作成してください。

要件

- 3×3の行列に対する以下の操作：
 - ポインターを使った要素アクセス
 - 行列の転置(ポインター使用)
 - 対角要素の合計
 - 行ごと、列ごとの合計

実装のヒント

- `*(arr + i) + j`でアクセス
- `int (*ptr)[3]`の使い方を理解

期待される出力例

```
=== 多次元配列とポインター ===
```

```
元の行列:
```

```
1 2 3
4 5 6
7 8 9
```

```
ポインターを使った要素アクセス:
```

```
*(*(matrix+1)+1) = 5
```

```
転置行列:
```

```
1 4 7
2 5 8
3 6 9
```

```
対角要素の合計: 15 (1 + 5 + 9)
```

```
行ごとの合計: 6, 15, 24
```

```
列ごとの合計: 12, 15, 18
```

問題9-5:動的配列の模擬

ポインターを使った配列の動的な操作を行うプログラムを作成してください。

要件

- 固定サイズ配列を使いながら動的配列のような操作：
 - 使用中の要素数を管理
 - 要素の追加(容量チェック付き)
 - 要素の削除(詰める処理)
 - 配列の拡張(別の大きな配列へコピー)

実装のヒント

- 実際の要素数と配列容量を別管理
- ポインターを使った効率的なコピー

期待される出力例

```
=== 動的配列の模擬 ===  
初期容量: 10, 使用中: 0  
  
要素を追加: 10, 20, 30  
現在の配列: [10, 20, 30]  
容量: 10, 使用中: 3  
  
要素 20 を削除...  
現在の配列: [10, 30]  
容量: 10, 使用中: 2  
  
要素を追加して容量オーバー時の処理:  
新しい容量: 20  
配列の内容をコピー中...  
拡張完了。
```

チャレンジ問題

問題9-6:高速文字列検索

ポインタを使った効率的な文字列検索アルゴリズムを実装してください。

要件

- Boyer-Moore法の簡易版を実装:
 - 文字列内から部分文字列を検索
 - 見つかった位置のポインタを返す
 - 複数箇所で見つかる場合はすべて表示

追加要件

- 検索回数をカウントして効率性を評価
- 通常の逐次検索と比較

期待される出力例

```
=== 高速文字列検索 ===  
テキスト: "The quick brown fox jumps over the lazy dog"  
検索パターン: "the"  
  
Boyer-Moore法 (簡易版):  
見つかった位置: 31 (0x7ffc200)  
検索回数: 12回  
  
通常の逐次検索:  
見つかった位置: 31  
検索回数: 32回  
  
効率性向上: 62.5%
```

問題9-7:メモリプールの実装

大きな配列を使ったメモリプールを実装してください。

要件

- 大きなchar配列をメモリプールとして使用
- 以下の機能を実装:
 - 指定サイズのメモリ割り当て
 - メモリの解放
 - 使用状況の表示
 - フラグメンテーションの管理

実装のヒント

- ・ ポインター演算でメモリブロックを管理
- ・ 各ブロックの使用状況をヘッダで管理

期待される出力例

```
=== メモリプール管理 ===
プールサイズ: 1024 バイト

メモリ割り当て:
ブロック1: 100/バイト確保 (アドレス: 0x7ffc000)
ブロック2: 200/バイト確保 (アドレス: 0x7ffc064)
ブロック3: 150/バイト確保 (アドレス: 0x7ffc12c)

使用状況:
使用中: 450/バイト (43.9%)
空き: 574/バイト (56.1%)

ブロック2を解放...
フラグメンテーション発生: 3個の断片

デフラグ実行...
連続した空き領域: 774/バイト
```

提出物

- ・ 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ・ ポインターを使う理由と、境界を保証する条件をコメントで記載してください。
- ・ 配列境界とNULLポインターを検査してください。

学習のポイント

- ・ 配列記法とポインター記法の変換を意識
- ・ ポインター演算の正確な理解
- ・ 関数への配列渡し仕組みを理解
- ・ メモリレイアウトを常に意識して実装

第10章:文字列の演習問題

演習の目的

- C言語の文字列(文字配列)の仕組みを深く理解する
- 標準ライブラリの文字列関数を使いこなす
- 安全な文字列操作の実装方法を習得する
- 文字列処理アルゴリズムを実装する

基礎問題

問題10-1:基本的な文字列操作

標準ライブラリの関数を使わずに、基本的な文字列操作関数を実装してください。

要件

- 以下の関数を実装:
 - `my_strlen()` - 文字列の長さを計算
 - `my_strcpy()` - 文字列をコピー
 - `my_strcmp()` - 文字列を比較
 - `my_strcat()` - 文字列を連結

実装のヒント

- ナル終端文字(‘\0’)の扱いに注意
- バッファオーバーフローを防ぐ

期待される出力例

```
=== 基本的な文字列操作 ===
文字列1: "Hello"
文字列2: "World"

my_strlen("Hello") = 5
my_strlen("World") = 5

my_strcpy実行後: "Hello"
my_strcmp("Hello", "World") = -15 (Hello < World)
my_strcmp("Hello", "Hello") = 0 (等しい)

my_strcat実行後: "HelloWorld"
```

問題10-2:文字列の検索と置換

文字列内の文字や部分文字列を検索・置換する関数を実装してください。

要件

- 以下の関数を実装:
 - `my_strchr()` - 文字を検索
 - `my_strrchr()` - 文字を後方検索
 - 特定文字の出現回数をカウント
 - 文字列内の特定文字を別の文字に置換

実装のヒント

- ポインターを使った効率的な実装
- NULLチェックを忘れずに

期待される出力例

```
=== 文字列の検索と置換 ===  
文字列: "Hello, World!"  
  
文字 'o' の検索:  
my_strchr: 最初の 'o' は位置 4  
my_strchr: 最後の 'o' は位置 7  
出現回数: 2回  
  
文字 'l' を 'L' に置換:  
置換後: "HeLLo, World!"
```

応用問題

問題10-3:文字列配列の管理

学生名を格納する文字列配列の管理システムを実装してください。

要件

- 最大20人の学生名を管理(各名前は最大50文字)
- 以下の機能を実装:
 - 学生名の追加
 - 学生名の削除
 - 学生名の検索
 - アルファベット順にソート
 - 全学生名の表示

実装のヒント

- 二次元配列を使用
- strcmp()でソートを実装

期待される出力例

```
=== 学生名管理システム ===

1. 学生追加
2. 学生削除
3. 学生検索
4. アルファベット順ソート
5. 全学生表示
6. 終了

選択: 1
学生名: Tanaka
追加しました。

選択: 1
学生名: Sato
追加しました。

選択: 5
--- 学生一覧 ---

1. Tanaka
2. Sato

選択: 4
ソート完了。

選択: 5
--- 学生一覧 ---

1. Sato
2. Tanaka
```

問題10-4:文字列の分割と結合

文字列を特定のデリミタで分割・結合する関数を実装してください。

要件

- カンマ区切りの文字列を分割して配列に格納
- 配列の文字列を指定のデリミタで結合
- 空白文字(スペース、タブ、改行)で分割
- トークンの前後の空白を除去(トリム機能)

実装のヒント

- strtok()の仕組みを理解
- 動的な配列管理を考慮

期待される出力例

```
=== 文字列の分割と結合 ===
元の文字列: "apple,banana,orange,grape"

カンマで分割:
1: "apple"
2: "banana"
3: "orange"
4: "grape"

スペースで結合: "apple banana orange grape"

空白文字で分割 (トリム付き) :
元の文字列: " hello world \t programming \n"
1: "hello"
2: "world"
3: "programming"
```

問題10-5:文字列の変換と検証

文字列の変換および検証を行う関数を実装してください。

要件

- 文字列を大文字/小文字に変換
- 文字列が数値として有効か検証
- 文字列を整数/浮動小数点数に変換
- パスワードの強度チェック(英数字、記号を含む8文字以上)

実装のヒント

- `ctype.h`の関数を活用
- エラー処理を適切に

期待される出力例

```
=== 文字列の変換と検証 ===
文字列: "Hello World 123"
大文字変換: "HELLO WORLD 123"
小文字変換: "hello world 123"

数値検証:
"123"  [ ] 有効な整数
"12.34" [ ] 有効な実数
"abc123" [ ] 無効な数値

文字列を数値に変換:
"123"  [ ] 123 (整数)
"45.67" [ ] 45.67 (実数)

パスワード強度チェック:
"password" [ ] 弱い (英字のみ)
"Pass123!" [ ] 強い (英数字と記号を含む8文字)
```

チャレンジ問題

問題10-6:高度な文字列検索

パターンマッチングアルゴリズムを実装してください。

要件

- ワイルドカード検索(*、?)の実装
- 大小文字を区別しない検索
- 単語境界での検索(単語単位の検索)
- 複数パターンの同時検索

追加要件

- 検索アルゴリズムの効率性を考慮
- KMP法やBoyer-Moore法の簡易版を実装

期待される出力例

```
=== 高度な文字列検索 ===
テキスト: "The quick brown fox jumps over the lazy dog"

ワイルドカード検索:
パターン "*fox*" [] マッチ (位置: 16)
パターン "qu?ck" [] マッチ (位置: 4)

大小文字を区別しない検索:
"THE" [] 見つかった位置: 0, 31

単語境界検索:
"the" [] 見つかった単語位置: 31 ("The"は単語境界でマッチしない)

複数パターン同時検索:
パターン: ["quick", "fox", "dog"]
結果:

- "quick": 位置 4
- "fox": 位置 16
- "dog": 位置 40
```

問題10-7:テキストエディターの基本機能

簡単なテキストエディターの基本機能を実装してください。

要件

- 複数行のテキストを管理(最大100行、各行最大80文字)
- 以下の機能を実装:
 - 行の挿入・削除
 - 文字列の検索・置換(全置換機能付き)
 - 行番号付きの表示
 - ファイルへの保存と読み込み

実装のヒント

- 二次元配列でテキストを管理
- ファイルI/Oの適切なエラー処理

期待される出力例

```
=== 簡易テキストエディター ===
```

1. 行の挿入
2. 行の削除
3. 検索
4. 置換
5. 表示
6. 保存
7. 読み込み
8. 終了

```
選択: 1  
行番号: 1  
内容: Hello, World!  
挿入しました。
```

```
選択: 5  
1: Hello, World!
```

```
選択: 3  
検索文字列: World  
見つかりました: 行1, 位置7
```

```
選択: 4  
検索文字列: World  
置換文字列: Programming  
全置換しますか(y/n): y  
1件置換しました。
```

```
選択: 5  
1: Hello, Programming!
```

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- バッファサイズと終端文字を検査し、入力エラー時の動作を実装してください。
- 警告を有効にしてコンパイルし、空文字列と上限長を含む実行結果を確認してください。

学習のポイント

- マル終端文字列の概念を確実に理解
- バッファサイズと文字列長の違いを意識
- ポインターと配列の関係を文字列で実践
- セキュアな文字列操作の重要性を認識

第11章:関数の演習問題

基礎問題

問題11-1:基本的な関数作成

以下の関数を作成してください。

1 数学関数

- 2つの整数の最大値を返す関数 `int max(int a, int b)`
- 3つの整数の最小値を返す関数 `int min(int a, int b, int c)`
- 数値が偶数かどうかを判定する関数 `int is_even(int n)`
- 数値が素数かどうかを判定する関数 `int is_prime(int n)`

2 文字・文字列関数

- 文字が英字かどうかを判定する関数 `int is_letter(char c)`
- 文字列の長さを計算する関数 `int my_strlen(const char str[])`
- 文字列内の特定文字を数える関数 `int count_char(const char str[], char ch)`

3 表示関数

- 指定した長さの線を描く関数 `void draw_line(int length, char ch)`
- 数値を指定した桁数で表示する関数 `void print_number_padded(int num, int width)`

期待される出力例

```
=== 数学関数のテスト ===
max(10, 20) = 20
min(30, 15, 25) = 15
is_even(24) = 1 (偶数)
is_even(17) = 0 (奇数)
is_prime(17) = 1 (素数)
is_prime(24) = 0 (素数ではない)

=== 文字・文字列関数のテスト ===
is_letter('A') = 1
is_letter('5') = 0
my_strlen("Hello") = 5
count_char("Hello World", 'l') = 3

=== 表示関数のテスト ===
draw_line(20, '-'): -----
print_number_padded(42, 5): 00042
print_number_padded(1234, 3): 1234
```

問題11-2:配列操作関数

以下の配列操作関数を作成してください。

1 基本操作

- 配列の要素の合計を計算する関数 `int array_sum(int arr[], int size)`
- 配列の平均値を計算する関数 `double array_average(int arr[], int size)`
- 配列内の最大値のインデックスを返す関数 `int find_max_index(int arr[], int size)`

2 検索・ソート

- 線形検索関数 `int linear_search(int arr[], int size, int target)`
- 配列を昇順にソートする関数 `void sort_array(int arr[], int size)`

- 配列の要素を逆順にする関数 `void reverse_array(int arr[], int size)`

3 配列統計

- 配列の最大値と最小値を同時に求める関数 `void find_min_max(int arr[], int size, int *min, int *max)`
- 配列内の重複する要素の数を数える関数 `int count_duplicates(int arr[], int size)`

期待される出力例

```
=== 配列操作関数のテスト ===
配列: [34, 12, 67, 89, 23, 45, 78, 56, 90, 11]

基本操作:
array_sum() = 495
array_average() = 49.50
find_max_index() = 8 (値: 90)

検索・ソート:
linear_search(45) = 5
ソート前: [34, 12, 67, 89, 23, 45, 78, 56, 90, 11]
ソート後: [11, 12, 23, 34, 45, 56, 67, 78, 89, 90]
逆順: [90, 89, 78, 67, 56, 45, 34, 23, 12, 11]

配列統計:
最小値: 11, 最大値: 90
重複要素数: 0

配列: [1, 2, 3, 2, 4, 3, 5]
重複要素数: 2 (2と3が重複)
```

問題11-3:文字列処理関数

以下の文字列処理関数を作成してください。

1 基本処理

- 文字列をコピーする関数 `void my_strcpy(char dest[], const char src[])`
- 文字列を連結する関数 `void my_strcat(char dest[], const char src[])`
- 文字列を比較する関数 `int my_strcmp(const char str1[], const char str2[])`

2 変換処理

- 文字列を大文字に変換する関数 `void to_uppercase(char str[])`
- 文字列を小文字に変換する関数 `void to_lowercase(char str[])`
- 文字列を逆順にする関数 `void reverse_string(char str[])`

3 解析処理

- 文字列内の単語数を数える関数 `int count_words(const char str[])`
- 文字列が回文かどうかを判定する関数 `int is_palindrome(const char str[])`
- 文字列内の母音の数を数える関数 `int count_vowels(const char str[])`

期待される出力例

```
=== 文字列処理関数のテスト ===
```

基本処理:

元の文字列: "Hello"

my_strcpy後: "Hello"

my_strcat後: "HelloWorld"

my_strcmp("Hello", "World") = -15

変換処理:

元: "Hello World"

大文字: "HELLO WORLD"

小文字: "hello world"

逆順: "dlroW olleH"

解析処理:

"Hello World Programming" の単語数: 3

"racecar" は回文: Yes

"hello" は回文: No

"Hello World" の母音数: 3 (e, o, o)

応用問題

問題11-4:複数戻り値を持つ関数

以下の関数を作成してください。

1 時間計算

- 秒数を時分秒に変換する関数 void seconds_to_hms(int total_seconds, int *hours, int *minutes, int *seconds)
- 日数から年月日に変換する関数(365日/年として) void days_to_ymd(int total_days, int *years, int *months, int *days)

2 座標計算

- 2点間の距離と中点を計算する関数 void calculate_line_info(double x1, double y1, double x2, double y2, double *distance, double *mid_x, double *mid_y)
- 円の面積と周囲の長さを計算する関数 void calculate_circle_info(double radius, double *area, double *circumference)

3 統計計算

- 配列の統計情報を計算する関数 void calculate_statistics(int arr[], int size, int *min, int *max, double *mean, double *median)
- 成績から評価を計算する関数 void calculate_grade_info(int scores[], int count, double *average, char *grade, int *pass_count)

期待される出力例

```
=== 複数戻り値関数のテスト ===

時間計算:
3661秒 = 1時間 1分 1秒
400日 = 1年 1月 5日

座標計算:
点(0, 0)と点(3, 4)の間:
距離: 5.00, 中点: (1.50, 2.00)

半径5の円:
面積: 78.54, 周囲: 31.42

統計計算:
配列: [34, 12, 67, 89, 23, 45, 78, 56, 90, 11]
最小: 11, 最大: 90, 平均: 50.50, 中央値: 50.50

成績情報:
点数: [85, 92, 78, 65, 98, 55, 73, 88]
平均: 79.25, 評価: B, 合格者数: 6
```

問題11-5:エラー処理付き関数

安全性を考慮した以下の関数を作成してください。

1 安全な基本操作

- 安全な除算関数 `int safe_divide(double a, double b, double *result)`
- 安全な配列アクセス関数 `int safe_array_get(int arr[], int size, int index, int *value)`
- 安全な文字列コピー関数 `int safe_strcpy(char dest[], int dest_size, const char src[])`

2 範囲チェック付き関数

- 範囲指定付きランダム数生成関数 `int random_range(int min, int max, int *result)`
- 配列の範囲チェック付き設定関数 `int safe_array_set(int arr[], int size, int index, int value)`

3 バリデーション関数

- 文字列が数値として有効かチェックする関数 `int is_valid_number(const char str[])`
- メールアドレスの基本形式をチェックする関数 `int is_valid_email(const char email[])`

期待される出力例

```
=== エラー処理関数のテスト ===

安全な除算:
safe_divide(10, 3) = 3.33 (成功)
safe_divide(10, 0) = エラー (ゼロ除算)

安全な配列アクセス:
safe_array_get(index=2) = 67 (成功)
safe_array_get(index=15) = エラー (範囲外)

安全な文字列コピー:
safe_strcpy("Hello") = 成功
safe_strcpy("VeryLongStringThatExceedsBufferSize") = エラー (バッファ不足)

バリデーション:
is_valid_number("123.45") = 1 (有効)
is_valid_number("12.3.4") = 0 (無効)
is_valid_email("user@example.com") = 1 (有効)
is_valid_email("invalid.email") = 0 (無効)
```

問題11-6:構造体を使った関数

以下の構造体を使った関数を作成してください。

```
typedef struct {
```

```

    double x;
    double y;
} Point;

typedef struct {
    int id;
    char name[50];
    int age;
    double score;
} Student;

typedef struct {
    int year;
    int month;
    int day;
} Date;

```

1 座標操作

- 2点間の距離を計算する関数 `double point_distance(Point p1, Point p2)`
- 点を移動する関数 `Point move_point(Point p, double dx, double dy)`
- 複数の点の重心を計算する関数 `Point calculate_centroid(Point points[], int count)`

2 学生情報処理

- 学生情報を表示する関数 `void print_student(Student s)`
- 学生の成績を更新する関数 `void update_score(Student *s, double new_score)`
- 学生配列から最高得点者を見つける関数 `Student find_best_student(Student students[], int count)`

3 日付操作

- 日付を表示する関数 `void print_date(Date d)`
- 日付の妥当性をチェックする関数 `int is_valid_date(Date d)`
- 2つの日付の差を計算する関数 `int date_difference(Date d1, Date d2)`

期待される出力例

```

=== 構造体関数のテスト ===

座標操作:
点A(0, 0)と点B(3, 4)の距離: 5.00
点(2, 3)を(1, -1)移動: (3, 2)
3点の重心: (2.00, 3.00)

学生情報処理:
学生: ID=1001, 名前=田中太郎, 年齢=20, 成績=85.5
成績更新後: 92.0
最高得点者: 山田花子 (成績: 95.0)

日付操作:
日付: 2024/3/15
is_valid_date(2024/2/30) = 0 (無効)
is_valid_date(2024/3/15) = 1 (有効)
2024/1/1 から 2024/3/15 までの日数: 74日

```

チャレンジ問題

問題11-7:再帰関数

以下の再帰関数を作成してください。

1 数学的再帰

- 階乗を計算する再帰関数 `long factorial_recursive(int n)`
- フィボナッチ数列を計算する再帰関数 `int fibonacci_recursive(int n)`

- ユークリッドの互除法による最大公約数を求める再帰関数 `int gcd_recursive(int a, int b)`

2 文字列再帰

- 文字列が回文かどうか再帰的に判定する関数 `int is_palindrome_recursive(const char str[], int start, int end)`
- 文字列を再帰的に逆順にする関数 `void reverse_string_recursive(char str[], int start, int end)`

3 配列再帰

- 配列の合計を再帰的に計算する関数 `int array_sum_recursive(int arr[], int size)`
- 配列の最大値を再帰的に見つける関数 `int find_max_recursive(int arr[], int size)`

期待される出力例

```
=== 再帰関数のテスト ===

数学的再帰:
5! = 120
fibonacci(10) = 55
gcd(48, 18) = 6

文字列再帰:
"racecar" は回文: Yes
"hello" は回文: No
"Hello" の逆順: "olleH"

配列再帰:
配列 [1, 2, 3, 4, 5] の合計: 15
配列 [34, 67, 12, 89, 23] の最大値: 89
```

問題11-8:高度な文字列処理

以下の高度な文字列処理関数を作成してください。

1 文字列解析

- 文字列をトークンに分割する関数 `int tokenize(char str[], char tokens[][50], char delimiter)`
- 文字列内の括弧の対応をチェックする関数 `int check_brackets(const char str[])`
- 文字列から数値を抽出する関数 `int extract_numbers(const char str[], int numbers[], int max_count)`

2 パターンマッチング

- 簡単なワイルドカード(*, ?)パターンマッチング関数 `int wildcard_match(const char str[], const char pattern[])`
- 文字列の置換関数 `int string_replace(char str[], const char old_substr[], const char new_substr[])`

3 文字列フォーマット

- 文字列を指定幅で中央揃えする関数 `void center_string(char result[], const char str[], int width)`
- CSV形式の文字列を解析する関数 `int parse_csv_line(const char line[], char fields[][100], int max_fields)`

問題11-9:ソートアルゴリズム関数

以下のソートアルゴリズムを関数として実装してください。

1 基本ソート

- バブルソート `void bubble_sort(int arr[], int size)`
- 選択ソート `void selection_sort(int arr[], int size)`
- 挿入ソート `void insertion_sort(int arr[], int size)`

2 高速ソート

- マージソート `void merge_sort(int arr[], int left, int right)`
- クイックソート `void quick_sort(int arr[], int left, int right)`

3 特殊ソート

- 文字列配列のソート `void sort_strings(char strings[][100], int count)`
- 構造体配列のソート(複数キー対応) `void sort_students(Student students[], int count, int sort_by)`

挑戦課題

課題11-10:関数ポインター

以下の関数ポインターを使った課題に取り組んでください。

1 計算機関数

- 四則演算を関数ポインターで切り替える計算機 `double calculator(double a, double b, double (*operation)(double, double))`
- 配列に対する処理を関数ポインターで指定する関数 `void process_array(int arr[], int size, void (*processor)(int*))`

2 ソート関数の汎用化

- 比較関数を引数に取る汎用ソート関数 `void generic_sort(void *arr, int size, int elem_size, int (*compare)(const void*, const void*))`

3 コールバック関数

- イベント処理システム `void register_callback(int event_type, void (*callback)(int))`

課題11-11:メモリ効率を考慮した関数

メモリ使用量を最適化した関数を作成してください。

1 動的メモリ管理

- 動的配列を管理する関数群
- 文字列プールを管理する関数群
- メモリリークを検出する関数

2 効率的なアルゴリズム

- インプレース(元の配列内で処理)ソート関数
- メモリ使用量を抑えた文字列処理関数
- キャッシュ効率を考慮した配列処理関数

課題11-12:総合プロジェクト

複数の関数を組み合わせた総合的なプロジェクトを作成してください。

1 学生管理システム

- 学生情報の登録、検索、更新、削除機能
- 成績統計の計算機能
- データのファイル保存・読み込み機能

2 テキスト解析ツール

- ファイルの読み込みと解析
- 単語頻度の統計
- 文字列パターンの検索

3 数値計算ライブラリ

- 行列演算機能
- 統計計算機能
- 数値積分・微分機能

提出物

- 関数を実装するCファイル、必要な宣言を置くヘッダーファイル、テスト用のmain関数を提出してください。
- 各関数の事前条件、戻り値、エラー時の動作を記載してください。
- 正常値、境界値、エラー値を含むテストを用意してください。

評価ポイント

- 正確性:仕様通りに動作するか
- 安全性:エラー処理が適切か
- 効率性:アルゴリズムの計算量は適切か
- 可読性:コードが理解しやすいか
- 再利用性:他のプログラムでも使えるか

難易度別の推奨学習順序

初学者向け(基本課題から開始)

- 1 問題11-1:基本的な関数作成
- 2 問題11-2:配列操作関数
- 3 問題11-3:文字列処理関数

中級者向け(中級課題に挑戦)

- 1 問題11-4:複数戻り値を持つ関数
- 2 問題11-5:エラー処理付き関数
- 3 問題11-6:構造体を使った関数

上級者向け(上級・挑戦課題)

- 1 問題11-7:再帰関数
- 2 問題11-8:高度な文字列処理
- 3 問題11-9:ソートアルゴリズム関数

- 4 課題11-10:関数ポインター
- 5 課題11-11:メモリ効率を考慮した関数
- 6 課題11-12:総合プロジェクト

学習のヒント

- 1 段階的実装
 - 最初は基本機能のみ実装
 - 動作確認後に機能を追加
 - テストを頻繁に実行
- 2 エラー処理
 - NULLポインターのチェック
 - 配列の境界チェック
 - 無効な引数の処理
- 3 関数設計の原則
 - 単一責任の原則
 - 適切な関数名と引数名
 - const修飾子の活用
- 4 テスト駆動開発
 - 関数作成前にテストケースを考える
 - 境界値のテスト
 - エラーケースのテスト
- 5 コードレビュー
 - 他の人に読んでもらう
 - 改善点を見つける
 - ベストプラクティスを学ぶ

参考資料

- C言語関数リファレンス
- アルゴリズムとデータ構造の教科書
- セキュアプログラミングガイド
- 関数型プログラミングの考え方

実装例の構成

各課題の実装では以下の構成を推奨します。

```
// ヘッダーファイル (functions.h)
// - 関数プロトタイプ
// - 定数定義
// - 構造体定義

// 実装ファイル (functions.c)
```

```
// - 関数の実装
// - 静的関数（内部使用）
// - エラー処理

// テストファイル（test.c）
// - main関数
// - 各関数のテスト
// - 結果の検証
```

発展的な学習内容

1 可変長引数

- va_list、va_start、va_endの使用
- printfライクな関数の作成

2 関数ポインター

- コールバック関数の実装
- 関数テーブルの作成

3 マクロ関数

- 関数ライクマクロの作成
- 条件付きコンパイル

4 インライン関数

- パフォーマンスの最適化
- 適切な使用場面

第12章:ビット操作の演習問題

演習問題の目的

この章では、ビット演算子の使い方、ビットマスクによるフラグ管理、ビットフィールドの活用、そして実践的なビット操作技術を習得します。

基礎問題

問題12-1:基本的なビット操作

以下の処理を行う関数を実装してください:

- 1 整数の特定ビットをセット/クリア/反転する関数
- 2 整数の特定範囲のビットを抽出する関数
- 3 整数のビットを左右に循環シフトする関数

```
/* 実装すべき関数のプロトタイプ */
void set_bit(unsigned int *value, int bit);
void clear_bit(unsigned int *value, int bit);
void toggle_bit(unsigned int *value, int bit);
int test_bit(unsigned int value, int bit);

unsigned int extract_bits(unsigned int value, int start, int count);
unsigned int rotate_left(unsigned int value, int shift);
unsigned int rotate_right(unsigned int value, int shift);
```

要件

- ビット位置は0から31まで
- 範囲外のビット位置の場合は何もしない
- 循環シフトは32ビットの範囲で行う

期待される出力例

```
=== 基本的なビット操作 ===
元の値: 0xA5 (10100101)

ビット3をセット: 0xAD (10101101)
ビット5をクリア: 0x85 (10000101)
ビット1を反転: 0xA7 (10100111)
ビット6のテスト: 1 (セットされている)

ビット2から4ビット抽出: 0x9 (1001)
左に3ビット循環シフト: 0x2D4 (1010110100)
右に2ビット循環シフト: 0x40000029 (0100000000000000000000000000010001)
```

問題12-2:フラグ管理システム

ゲームキャラクターの状態を管理するフラグシステムを実装してください:

```
/* 状態フラグの定義 */
#define STATUS_ALIVE 0x0001
#define STATUS_POISONED 0x0002
#define STATUS_PARALYZED 0x0004
#define STATUS_CONFUSED 0x0008
#define STATUS_SLEEPING 0x0010
#define STATUS_BURNING 0x0020
#define STATUS_FROZEN 0x0040
#define STATUS_INVISIBLE 0x0080
#define STATUS_INVINCIBLE 0x0100
#define STATUS_BUFFED 0x0200
```

実装する機能:

- 1 複数の状態を同時に設定/解除

- 2 特定の状態の組み合わせをチェック
- 3 状態の一覧を表示
- 4 相反する状態の自動解除(例:凍結と燃焼)

期待される出力例

```
=== キャラクター状態管理 ===  
初期状態: ALIVE  
  
状態を追加: POISONED, CONFUSED  
現在の状態: ALIVE | POISONED | CONFUSED  
  
BURNINGを追加...  
FROZENが自動的に解除されました  
現在の状態: ALIVE | POISONED | CONFUSED | BURNING  
  
状態チェック:  
  
- 生存している: はい  
- 毒と混乱の両方: はい  
- 無敵状態: いいえ  
  
全状態一覧:  
[x] ALIVE  
[x] POISONED  
[ ] PARALYZED  
[x] CONFUSED  
[ ] SLEEPING  
[x] BURNING  
[ ] FROZEN  
[ ] INVISIBLE  
[ ] INVINCIBLE  
[ ] BUFFED
```

応用問題

問題12-3:ビットフィールドでメモリ節約

学生の成績データを効率的に格納する構造体を設計してください:

```
/* 成績データ構造体 (ビットフィールド使用) */  
struct StudentGrade {  
    unsigned int student_id : 20; /* 学生ID (0-1048575) */  
    unsigned int subject_code : 10; /* 科目コード (0-1023) */  
    unsigned int grade : 7; /* 成績 (0-100) */  
    unsigned int semester : 2; /* 学期 (1-4) */  
    unsigned int year : 7; /* 年度 (0-99, 2000年基準) */  
    unsigned int attendance : 1; /* 出席状況 (合格/不合格) */  
    unsigned int retake : 1; /* 再履修フラグ */  
};
```

実装する機能:

- 1 成績データの入力と表示
- 2 通常の構造体とのサイズ比較
- 3 成績の統計情報(平均点、最高点、最低点)
- 4 条件に合う学生の検索

期待される出力例

=== 成績データ管理 ===
構造体サイズ比較:

- ビットフィールド版: 8 バイト
- 通常版: 32 バイト
- 節約率: 75.0%

成績データ入力:

学生ID: 12345
科目コード: 101
成績: 85
学期: 2
年度: 24 (2024年)
出席: 合格
再履修: いいえ

登録された成績一覧:

ID	科目	成績	学期	年度	出席	再履修
12345	101	85	2	2024	yes	no
12346	101	92	2	2024	yes	no
12347	101	68	2	2024	no	yes

統計情報:

平均点: 81.7
最高点: 92 (学生ID: 12346)
最低点: 68 (学生ID: 12347)

問題12-4:RGB色操作

RGB565形式(16ビット)の色を操作するプログラムを作成してください:

```
/* RGB565色構造体 */
typedef struct {
    unsigned int blue : 5;
    unsigned int green : 6;
    unsigned int red : 5;
} RGB565;

/* または16ビット整数として扱う */
typedef unsigned short RGB565_int;
```

実装する機能:

- 1 RGB888(24ビット)⇔ RGB565の相互変換
- 2 色の明度調整(明るく/暗く)
- 3 グレースケール変換
- 4 2つの色の混合(アルファブレンド風)

期待される出力例

=== RGB565色操作 ===
RGB888 (255, 128, 64) → RGB565: 0xFC20
RGB565: R=31, G=32, B=0

RGB565 0xFC20 → RGB888: (248, 129, 0)
(精度損失あり)

明度調整:

元の色: 0x7BEF (R=15, G=31, B=15)
50%明るく: 0xBDF7 (R=23, G=47, B=23)
50%暗く: 0x39E7 (R=7, G=15, B=7)

グレースケール変換:

カラー: 0xF800 (赤) → グレー: 0x7BEF

色の混合:

色1: 0xF800 (赤)
色2: 0x001F (青)
混合(50:50): 0x780F (紫)

問題12-5:ビットベクタで集合演算

ビットベクタを使って集合演算を実装してください:

```
#define SET_SIZE 256 /* 0-255の整数を扱う */

typedef struct {
    unsigned char bits[SET_SIZE / 8];
} BitSet;
```

実装する機能:

- 1 集合の初期化、要素の追加/削除
- 2 和集合、積集合、差集合、対称差
- 3 部分集合の判定
- 4 集合の要素数カウント
- 5 集合の内容表示

期待される出力例

```
=== ビットベクタ集合演算 ===
集合A: {1, 3, 5, 7, 9}
集合B: {2, 3, 5, 8, 9}

和集合 (A ∪ B): {1, 2, 3, 5, 7, 8, 9}
積集合 (A ∩ B): {3, 5, 9}
差集合 (A - B): {1, 7}
対称差 (A ⊕ B): {1, 2, 7, 8}

集合C: {3, 5}
C ⊆ A: true (CはAの部分集合)
C ⊆ B: true (CはBの部分集合)

要素数:
|A| = 5
|B| = 5
|A ∪ B| = 7
|A ∩ B| = 3
```

チャレンジ問題

問題12-6:CRC計算の実装

簡単なCRC-8チェックサムを計算する関数を実装してください:

```
/* CRC-8計算 (多項式:  $x^8 + x^2 + x + 1$ ) */
unsigned char calculate_crc8(const unsigned char *data, int length);
```

実装内容:

- 1 ビット単位でCRCを計算する基本実装
- 2 テーブルを使った高速化実装
- 3 データの破損検出テスト
- 4 性能比較

期待される出力例

```
=== CRC-8計算 ===
データ: "Hello"
CRC-8 (ビット単位): 0x1A
CRC-8 (テーブル): 0x1A

破損検出テスト:
元データ: "Hello" (CRC: 0x1A)
破損データ: "Hallo" (CRC: 0x4D)
破損が検出されました。

性能比較 (10000回実行):
ビット単位: 125ms
テーブル方式: 15ms
高速化: 8.3倍
```

問題12-7:ビットボードでオセロ

8×8のオセロ盤面をビットボードで表現してください:

```
typedef struct {
    unsigned long long black; /* 黒石の位置 */
    unsigned long long white; /* 白石の位置 */
} OthelloBoard;
```

実装する機能:

- 1 盤面の初期化と表示
- 2 石を置ける位置の判定
- 3 石を置いた時の反転処理
- 4 勝敗判定

期待される出力例

```
=== オセロゲーム ===
 A B C D E F G H
1 . . . . .
2 . . . . .
3 . . * . . .
4 . . * O X . .
5 . . X O * . .
6 . . . * . . .
7 . . . . .
8 . . . . .

黒番です
置ける場所(*): D3, C4, F5, E6

D3に置きます...

 A B C D E F G H
1 . . . . .
2 . . . . .
3 . . X . . .
4 . . * X X . .
5 . . X O * . .
6 . . . * . . .
7 . . . . .
8 . . . . .

石の数: 黒=4, 白=1
```

問題12-8:ハフマン符号化(上級)

簡単なハフマン符号化を実装してください:

- 1 文字の出現頻度をカウント
- 2 ハフマン木を構築

- 3 各文字のビット列を決定
- 4 テキストをビット列に圧縮
- 5 ビット列から元のテキストに復元

問題12-9:ブルームフィルター(上級)

確率的データ構造であるブルームフィルタを実装してください:

- 1 複数のハッシュ関数の実装
- 2 要素の追加
- 3 要素の存在確認(偽陽性あり)
- 4 偽陽性率の測定

演習のヒント

ビット操作の基本パターン

```
/* nビット目をセット */
value |= (1 << n);

/* nビット目をクリア */
value &= ~(1 << n);

/* nビット目を反転 */
value ^= (1 << n);

/* nビット目をテスト */
if (value & (1 << n)) { /* ... */ }
```

ビットマスクの作成

```
/* 下位nビットのマスク */
mask = (1 << n) - 1;

/* startからcountビット分のマスク */
mask = ((1 << count) - 1) << start;
```

ビットカウント

```
/* Brian Kernighanのアルゴリズム */
int count = 0;
while (n) {
    n &= n - 1;
    count++;
}
```

提出物

- 問題ごとにex12_1.c、ex12_2.cのような名前でCファイルを作成してください。
- 各ファイルに動作確認用のmain関数を含めてください。
- ビットマスクの設計と境界条件をコメントで説明してください。

評価基準

- **正確性:** 要求された機能が正しく動作するか
- **効率性:** ビット操作の利点を活かした実装か
- **可読性:** コードが理解しやすく整理されているか
- **完成度:** エラー処理やエッジケースへの対応

第13章:構造体の演習問題

演習の目的

- 構造体の定義と使い方を理解する
- 構造体のメンバーアクセス(ドット演算子とアロー演算子)を習得する
- 構造体配列と構造体ポインタの操作を身につける
- ネストした構造体の設計と実装を経験する

基礎問題

問題13-1:学生情報管理

学生の情報(ID、名前、年齢、成績)を格納する構造体を定義し、ポインタを使って情報を表示・更新するプログラムを作成してください。

要件

- 学生情報を表す構造体を定義
- 学生情報を入力する関数を作成
- 学生情報を表示する関数を作成(ポインタ引数)
- 成績を更新する関数を作成(ポインタ引数)

実装のヒント

- 構造体ポインタのメンバーアクセスには->演算子を使用
- 文字列のコピーにはstrcpy()を使用

期待される出力例

```
=== 学生情報管理システム ===
学生情報を入力してください:
ID: 1001
名前: 山田太郎
年齢: 20
成績: 85.5

入力された学生情報:
ID: 1001
名前: 山田太郎
年齢: 20歳
成績: 85.50点

成績を更新します...
新しい成績: 92.0

更新後の学生情報:
ID: 1001
名前: 山田太郎
年齢: 20歳
成績: 92.00点
```

問題13-2:座標計算

2D座標を表す構造体を定義し、2点間の距離を計算する関数をポインタを使って実装してください。

要件

- 座標を表す構造体(x, y)を定義
- 2点間の距離を計算する関数を作成(ポインタ引数)

- 座標を移動する関数を作成(ポインターで更新)
- math.hのsqrt関数を使用

実装のヒント

- 距離の公式: $\text{sqrt}((x_2-x_1)^2 + (y_2-y_1)^2)$
- コンパイル時に-lmオプションが必要

期待される出力例

```
=== 座標計算プログラム ===
点A の座標:
x: 0
y: 0

点B の座標:
x: 3
y: 4

点A(0.00, 0.00) と 点B(3.00, 4.00) の距離: 5.00

点Aを移動します:
x方向の移動量: 2
y方向の移動量: 1

移動後の点A: (2.00, 1.00)
新しい距離: 2.24
```

問題13-3:商品管理

商品情報(コード、名前、価格、在庫)の構造体を作成し、構造体配列で複数商品を管理するプログラムを作成してください。

要件

- 商品情報を表す構造体を定義
- 最大10個の商品を管理できる配列を用意
- 商品を追加する関数を作成
- 全商品を表示する関数を作成
- 在庫を更新する関数を作成(商品コードで検索)

実装のヒント

- 構造体配列の初期化に注意
- 商品数のカウンタを別途管理

期待される出力例

```
=== 商品管理システム ===
```

1. 商品追加
2. 商品一覧表示
3. 在庫更新
4. 終了

```
選択: 1  
商品コード: P001  
商品名: ノート  
価格: 150  
在庫数: 50  
商品を追加しました。
```

```
選択: 1  
商品コード: P002  
商品名: ペン  
価格: 100  
在庫数: 100  
商品を追加しました。
```

```
選択: 2  
--- 商品一覧 ---  
コード: P001  
商品名: ノート  
価格: 150円  
在庫: 50個
```

```
コード: P002  
商品名: ペン  
価格: 100円  
在庫: 100個
```

```
選択: 3  
更新する商品コード: P001  
新しい在庫数: 45  
在庫を更新しました。
```

応用問題

問題13-4:従業員データベース

従業員情報と部署情報をネストした構造体で管理し、部署別の給与統計を算出するプログラムを作成してください。

要件

- 部署情報を表す構造体(部署名、部署コード)
- 従業員情報を表す構造体(ID、名前、部署情報、給与)
- 部署別の平均給与を計算する関数
- 最高給与の従業員を検索する関数
- 構造体ポインター配列を使用

実装のヒント

- ネストした構造体のアクセス方法を理解
- 部署コードで従業員をグループ化

期待される出力例

```
=== 従業員データベース ===
従業員データ:
ID: 1001, 名前: 田中太郎
部署: 営業部 (コード: SALES)
給与: 350000円

ID: 1002, 名前: 鈴木花子
部署: 営業部 (コード: SALES)
給与: 320000円

ID: 1003, 名前: 佐藤次郎
部署: 開発部 (コード: DEV)
給与: 450000円

=== 部署別統計 ===
営業部 (SALES):

- 人数: 2名
- 平均給与: 335000円

開発部 (DEV):

- 人数: 1名
- 平均給与: 450000円

最高給与の従業員:
佐藤次郎 (開発部) - 450000円
```

問題13-5:図書管理システム

本の情報(タイトル、著者、出版年、貸出状況)を管理し、検索・貸出・返却機能を実装してください。

要件

- 書籍情報を表す構造体を定義
- 貸出状況を管理(貸出中フラグ、借りた人のID)
- タイトルで検索する関数
- 貸出処理を行う関数
- 返却処理を行う関数
- 貸出中の本一覧を表示する関数

実装のヒント

- 部分一致検索には`strstr()`を使用
- 貸出状況は構造体内のフラグで管理

期待される出力例

```
=== 図書管理システム ===

1. 図書検索
2. 貸出処理
3. 返却処理
4. 貸出中一覧
5. 終了

選択: 1
検索するタイトル: プログラミング

検索結果:

1. 「C言語プログラミング入門」 - 山田太郎 (2023年)

    状態: 貸出可能

2. 「プログラミング言語C」 - カーニハン&リッチー (1988年)

    状態: 貸出中 (借主ID: U001)

選択: 2
貸出する本の番号: 1
借りる人のID: U002
「C言語プログラミング入門」を貸出しました。

選択: 4
--- 貸出中の本一覧 ---

1. 「プログラミング言語C」

    借主ID: U001

2. 「C言語プログラミング入門」

    借主ID: U002
```

チャレンジ問題

問題13-6:成績管理システム

学生と科目の構造体を使って、学生別・科目別の成績統計を管理するプログラムを作成してください。

要件

- 科目情報を表す構造体(科目名、科目コード、単位数)
- 成績情報を表す構造体(学生ID、科目コード、点数、評価)
- 学生別の平均点を計算する関数
- 科目別の平均点を計算する関数
- GPA計算機能(A=4.0, B=3.0, C=2.0, D=1.0, F=0.0)

追加要件

- CSVファイルからのデータ読み込み機能
- 成績分布のヒストグラム表示

期待される出力例

```
=== 成績管理システム ===
データ読み込み完了: 15件

学生別統計:
学生ID 1001:

- 受講科目数: 3
- 平均点: 83.3
- GPA: 3.33

科目別統計:
プログラミング基礎 (CS101):

- 受講者数: 5
- 平均点: 78.4
- 成績分布:

  A (90-100): ** (2名)
  B (80-89): * (1名)
  C (70-79): * (1名)
  D (60-69): * (1名)
  F (0-59): - (0名)

全体GPA順位:

1. 学生ID 1003: GPA 3.80
2. 学生ID 1001: GPA 3.33
3. 学生ID 1002: GPA 2.95
```

問題13-7:連結リストの実装

構造体を使って単方向連結リストを実装してください。

要件

- ノード構造体(データと次のノードへのポインター)
- リストの先頭に要素を追加
- リストの末尾に要素を追加
- 特定の値を持つノードを削除
- リスト全体を表示
- メモリリークを防ぐ適切な解放処理

実装のヒント

- 自己参照構造体の定義方法を理解
- malloc/freeを使った動的メモリ管理

期待される出力例

```
=== 連結リスト操作 ===

1. 先頭に追加
2. 末尾に追加
3. 削除
4. 表示
5. 終了

選択: 1
値: 10
先頭に10を追加しました。

選択: 2
値: 20
末尾に20を追加しました。

選択: 1
値: 5
先頭に5を追加しました。

選択: 4
リスト: 5 -> 10 -> 20 -> NULL

選択: 3
削除する値: 10
10を削除しました。

選択: 4
リスト: 5 -> 20 -> NULL

選択: 5
メモリを解放しています...
終了しました。
```

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- 構造体のメンバー構成と関数へ渡す方法をコメントで説明してください。
- 動的メモリを使用する場合は、確保と解放を対応させてください。

学習のポイント

- 構造体は関連するデータをまとめる強力な機能
- ポインター経由のアクセスでは->演算子を使用
- 大きな構造体や変更対象の構造体は、必要に応じてポインターで渡す
- ネストした構造体で複雑なデータ構造を表現可能

第14章:関数ポインターの演習問題

演習の目的

- 関数ポインターの基本概念を理解する
- コールバック関数の実装方法を習得する
- 関数ポインター配列を活用した動的関数選択を学ぶ
- 実行時関数切り替えシステムの設計を理解する

基礎問題

問題14-1:関数ポインターの基本操作

関数ポインターを使って複数の関数を動的に呼び出すプログラムを作成してください。

要件

- 2つの整数を受け取り、結果を返す関数を複数定義する
- 関数ポインターを使ってこれらの関数を呼び出す
- 関数ポインター配列を使った実装も含める

実装すべき関数

- int maximum(int a, int b) - 大きい方の値を返す
- int minimum(int a, int b) - 小さい方の値を返す
- int power(int a, int b) - aのb乗を返す(簡単な実装で可)

期待される出力例

```
=== 関数ポインターの基本操作 ===
a = 10, b = 5

関数ポインターで直接呼び出し:
maximum(10, 5) = 10
minimum(10, 5) = 5
power(10, 5) = 100000

関数ポインター配列で呼び出し:
操作0: maximum(10, 5) = 10
操作1: minimum(10, 5) = 5
操作2: power(10, 5) = 100000

関数アドレス表示:
maximum: 0x400650
minimum: 0x400670
power: 0x400690
```

ファイル名: ex14_1_basic_function_pointer.c

問題14-2:関数選択システム

文字に基づいて関数を選択し実行するシステムを実装してください。

要件

- 文字('+', '-', '*', '/', ...)に基づいて関数を選択
- 選択された関数を実行して結果を表示
- 無効な文字が指定された場合のエラー処理
- 構造体を使った関数ポインター管理

期待される出力例

```
=== 関数選択システム ===  
利用可能な演算: + - * / % ^  
  
第1の数: 15  
第2の数: 3  
演算子: +  
結果: 15 + 3 = 18  
  
第1の数: 20  
第2の数: 4  
演算子: /  
結果: 20 / 4 = 5  
  
第1の数: 10  
第2の数: 0  
演算子: /  
エラー: ゼロ除算はできません  
  
第1の数: 8  
第2の数: 3  
演算子: @  
エラー: 無効な演算子です
```

ファイル名: ex14_2_function_selector.c

応用問題

問題14-3:配列処理のコールバック

コールバック関数を使って配列の各要素に異なる処理を適用するプログラムを作成してください。

要件

- 整数配列を処理する関数を複数定義
- コールバック関数として配列処理関数に渡す
- 処理前後の配列の状態を表示
- 動的な処理選択機能

実装すべき処理

- 各要素を倍にする
- 各要素から1を引く
- 各要素の符号を反転する

期待される出力例

```
=== 配列処理のコールバック ===
元の配列: [1, 2, 3, 4, 5]

処理選択:

1. 各要素を倍にする
2. 各要素から1を引く
3. 各要素の符号を反転

選択: 1

処理後: [2, 4, 6, 8, 10]

別の配列: [5, -3, 8, -2, 0]
処理選択: 3

処理後: [-5, 3, -8, 2, 0]

複数処理の連続実行:
初期配列: [10, 20, 30]
□ 倍にする: [20, 40, 60]
□ 1を引く: [19, 39, 59]
最終結果: [19, 39, 59]
```

ファイル名: ex14_3_array_callback.c

問題14-4:関数ポインター配列を使った計算機

関数ポインター配列を使用した計算機プログラムを作成してください。

要件

- 関数ポインターの配列を定義
- インデックスを指定して演算を選択
- 複数の演算を連続で実行可能
- 演算履歴を表示する機能
- 統計情報の収集と表示

実装すべき演算

- 加算、減算、乗算、除算
- ゼロ除算のエラー処理

期待される出力例

```
=== 関数ポインター配列計算機 ===
演算メニュー：
0: 加算 (+)
1: 減算 (-)
2: 乗算 (*)
3: 除算 (/)
4: 終了

第1の数: 100
第2の数: 25
演算番号: 0
結果: 100 + 25 = 125

演算番号: 3
結果: 100 / 25 = 4

=== 演算履歴 ===

1. 100 + 25 = 125
2. 100 / 25 = 4

=== 統計情報 ===
総演算回数: 2

- 加算: 1回 (50.0%)
- 減算: 0回 (0.0%)
- 乗算: 0回 (0.0%)
- 除算: 1回 (50.0%)
```

ファイル名: ex14_4_calculator_function_array.c

チャレンジ問題

問題14-5:ソートアルゴリズム選択システム

異なるソートアルゴリズムを関数ポインターで切り替えるシステムを作成してください。

要件

- バブルソート、選択ソート、挿入ソートを実装
- 比較関数も関数ポインターで指定(昇順/降順)
- ソート前後の配列状態を表示
- アルゴリズムの性能比較機能

期待される出力例

```
=== ソートアルゴリズム選択システム ===
配列: [34, 7, 23, 32, 5, 62]

ソートアルゴリズム:

1. バブルソート
2. 選択ソート
3. 挿入ソート

選択: 1

順序:

1. 昇順
2. 降順

選択: 1

バブルソート（昇順）実行中...
結果: [5, 7, 23, 32, 34, 62]
比較回数: 15, 交換回数: 8

=== 性能比較（配列サイズ: 100） ===
アルゴリズム   比較回数   交換回数   時間(ms)
バブルソート    4950       2475       0.125
選択ソート      4950        99        0.098
挿入ソート      2525       2426       0.087
```

ファイル名: ex14_5_sort_algorithms.c

問題14-6:イベント駆動システム

コールバック関数を使ったイベント駆動システムを実装してください。

要件

- 複数種類のイベント(開始、停止、エラー、警告)
- イベントタイプごとに異なるハンドラーを登録
- イベント発生時に適切なハンドラーを呼び出し
- イベントの優先度機能

期待される出力例

```
=== イベント駆動システム ===
イベントハンドラーを登録中...

イベント発生シミュレーション:

[開始] システムが起動しました
[警告] メモリ使用率が80%を超えました
[エラー] ファイルが見つかりません: config.txt
[停止] システムをシャットダウンします

優先度付きイベント処理:
優先度3: [エラー] 重大なエラーが発生
優先度1: [警告] 軽微な警告
優先度2: [開始] サービス開始

=== イベント統計 ===
開始イベント: 2回
停止イベント: 1回
エラーイベント: 2回
警告イベント: 2回
総イベント数: 7回
```

ファイル名: ex14_6_event_system.c

提出物

- 問題ごとにC90版とC99版を作成し、C99版には_c99.cを付けてください。
- ファイルの冒頭に問題番号と関数ポインターの型をコメントで記載してください。
- それぞれの規格を指定し、警告を有効にしてコンパイルしてください。

コンパイル例

C90準拠

```
gcc -std=c90 -Wall -Wextra -pedantic ex14_1_basic_function_pointer.c -o ex14_1_basic_function_pointer
```

C99準拠

```
gcc -std=c99 -Wall -Wextra -pedantic ex14_1_basic_function_pointer_c99.c -o ex14_1_basic_function_pointer_c99
```

学習のポイント

関数ポインターの基本

- 1 宣言: `int (*func_ptr)(int, int);`
- 2 初期化: `func_ptr = function_name;`
- 3 呼び出し: `result = func_ptr(a, b);`
- 4 配列: `int (*operations[])(int, int) = {add, sub, mul};`

コールバック関数

- 1 概念: 関数を引数として渡す仕組み
- 2 活用: 動的な処理選択、カスタマイズ可能な処理
- 3 設計: 関数ポインターを引数に取る関数の実装

実用的なテクニック

- 1 関数テーブル: 構造体による関数ポインター管理
- 2 エラー処理: 無効な関数ポインターの検出
- 3 状態管理: 関数ポインターによる状態遷移
- 4 性能: 関数ポインター使用時のオーバーヘッド考慮

注意事項

- 関数ポインターは初期化前に使用しない
- 型の一致を厳密に確認する
- NULLポインターチェックを忘れずに
- デバッグ時は関数名の確認が困難な場合がある

実用的な応用

これらの技術は以下のような場面で使用されます。

- GUIフレームワーク: イベントハンドラ
- 組み込みシステム: 割り込みハンドラ
- ゲーム開発: 状態管理、AI行動パターン

- ・ システムプログラミング:プラグインシステム

第15章:分割コンパイルと発展技術の演習問題

演習の目的

- プリプロセッサの高度な使い方を習得する
- メモリ管理の最適化技術を学ぶ
- キャッシュ効率を考慮したプログラミングを理解する
- システムプログラミングの基礎を身につける

基礎問題

問題15-1:プリプロセッサマクロの基本

以下の仕様を満たすプログラムを作成してください。

要件

- 数学的な定数をマクロで定義する(PI、E、黄金比など)
- 基本的な計算マクロを定義する(面積、体積など)
- 条件付きコンパイルでデバッグ機能を切り替える
- 文字列化マクロと連結マクロを活用する

実装すべき機能

- 円の面積・周長計算
- 球の体積・表面積計算
- デバッグ情報の出力(条件付き)
- マクロによる型安全な最大値・最小値関数

期待される出力例

```
=== プリプロセッサマクロの基本 ===
円の計算 (半径 = 5.0) :

- 面積: 78.54
- 周長: 31.42

球の計算 (半径 = 3.0) :

- 体積: 113.10
- 表面積: 113.10

[DEBUG] 計算完了: 円の面積 = 78.54 (半径: 5.0)
[DEBUG] 計算完了: 球の体積 = 113.10 (半径: 3.0)

型安全な最大値・最小値:
MAX(10, 20) = 20
MIN(3.14, 2.71) = 2.71

黄金比を使った計算:
黄金長方形 (短辺 = 10): 長辺 = 16.18
```

ファイル名: ex15_1_macro_basics.c

問題15-2:安全なメモリ操作マクロ

メモリ操作の安全性を向上させるマクロ群を作成してください。

要件

- NULLチェック付きメモリ割り当て

- 配列境界チェック付きアクセス
- 自動的なメモリ解放
- メモリリーク検出機能

実装すべきマクロ

- `SAFE_MALLOC(size)` - NULLチェック付きmalloc
- `SAFE_FREE(ptr)` - NULLクリア付きfree
- `ARRAY_BOUNDS_CHECK(arr, index, size)` - 境界チェック
- `MEMORY_LEAK_TRACKER` - 簡単なリーク検出

期待される出力例

```

=== 安全なメモリ操作マクロ ===

通常のメモリ割り当て:
1000バイトを割り当て: 成功 (0x7fff5fbff8c0)
解放完了 (ポインタはNULLにリセット)

メモリ不足シミュレーション:
SAFE_MALLOC失敗: メモリ割り当てエラー (行: 45)

配列境界チェック:
配列[0-9]へのアクセステスト:
index 5: OK (値: 50)
index 10: エラー - 境界外アクセス検出。 (行: 67)

=== メモリリーク検出レポート ===
総割り当て: 1024 バイト
総解放: 768 バイト
リーク検出: 256 バイト (1ブロック)

- ブロック#3: 256バイト (行: 89で割り当て)

```

ファイル名: `ex15_2_safe_memory.c`

問題15-3:基本的なメモリプール

固定サイズオブジェクト用の簡単なメモリプールを実装してください。

要件

- 事前に確保されたメモリ領域からオブジェクトを割り当て
- フリーリストによる高速な割り当て・解放
- プールの使用状況を表示する機能
- 初期化・終了処理

実装すべき機能

- プールの初期化と終了処理
- オブジェクトの取得と返却
- 使用状況の表示
- エラー処理

期待される出力例

```
=== 基本的なメモリプール ===
プール初期化: 100個のオブジェクト (各64バイト)

10個のオブジェクトを割り当て...
割り当て成功: 10個

=== プール使用状況 ===
総容量: 100 オブジェクト
使用中: 10 オブジェクト (10.0%)
空き: 90 オブジェクト (90.0%)
総メモリ: 6400 バイト

5個のオブジェクトを返却...

=== 更新後の使用状況 ===
使用中: 5 オブジェクト (5.0%)
空き: 95 オブジェクト (95.0%)

パフォーマンステスト (1000回の割り当て/解放):
- メモリプール: 0.15ms
- 通常のmalloc/free: 2.34ms
- 高速化: 15.6倍
```

ファイル名: ex15_3_memory_pool.c

応用問題

問題15-4:汎用的なプリプロセッサライブラリ

さまざまな用途に使える汎用的なマクロライブラリを作成してください。

要件

- 型判定マクロ(コンパイル時)
- 汎用的なスワップマクロ
- ループ展開マクロ
- アサーション機能付きマクロ
- ベンチマーク計測マクロ

実装すべき機能

- TYPE_CHECK(a, b) - 型の一致確認
- GENERIC_SWAP(a, b) - 任意の型のスワップ
- REPEAT(n, code) - コードの繰り返し展開
- BENCHMARK_BLOCK(name) - ブロックの実行時間測定
- STATIC_ASSERT(condition, message) - コンパイル時アサーション

期待される出力例

```
=== 汎用的なプリプロセッサライブラリ ===

型チェック:
TYPE_CHECK(int, int): OK
TYPE_CHECK(int, double): 型不一致 (コンパイル時警告)

汎用スワップ:
整数: a=10, b=20 → a=20, b=10
文字列: s1="Hello", s2="World" → s1="World", s2="Hello"
構造体: p1=(1,2), p2=(3,4) → p1=(3,4), p2=(1,2)

ループ展開 (5回):
展開0: i=0, value=0
展開1: i=1, value=1
展開2: i=2, value=4
展開3: i=3, value=9
展開4: i=4, value=16

=== ベンチマーク結果 ===
[BENCHMARK] 配列初期化: 0.234ms
[BENCHMARK] 行列演算: 1.567ms
[BENCHMARK] ソート処理: 3.890ms

スタティックアサート:
コンパイル時チェック: すべてOK
```

ファイル名: ex15_4_generic_macros.c

問題15-5:高性能メモリアロケーター

パフォーマンスを重視したカスタムメモリアロケーターを実装してください。

要件

- サイズ別メモリプール(小・中・大オブジェクト)
- メモリの断片化を最小限に抑える仕組み
- アロケーション統計情報の収集
- スレッドセーフ対応(簡易版)

実装すべき機能

- 複数サイズのメモリプール管理
- First Fit / Best Fitアルゴリズム
- メモリ使用統計とレポート機能
- デバッグモードでの詳細トレース

期待される出力例

```
=== 高性能メモリアロケータ ===

アロケータを初期化中...

- 小オブジェクトプール: 0-64 バイト
- 中オブジェクトプール: 65-512 バイト
- 大オブジェクトプール: 513+ バイト

割り当てテスト:
32バイト割り当て: 成功 (小プールから)
256バイト割り当て: 成功 (中プールから)
1024バイト割り当て: 成功 (大プールから)

=== メモリ使用統計 ===
小オブジェクトプール:
  割り当て回数: 150
  解放回数: 120
  現在使用中: 30 (1920 バイト)
  断片化率: 5.2%

中オブジェクトプール:
  割り当て回数: 50
  解放回数: 45
  現在使用中: 5 (1280 バイト)
  断片化率: 8.7%

大オブジェクトプール:
  割り当て回数: 10
  解放回数: 8
  現在使用中: 2 (2048 バイト)
  断片化率: 12.3%

総メモリ使用量: 5248 バイト
平均割り当て時間: 0.0034ms
```

ファイル名: ex15_5_allocator.c

チャレンジ問題

問題15-6: キャッシュフレンドリーなデータ構造

CPUキャッシュ効率を考慮したデータ構造を実装してください。

要件

- 配列ベースの動的配列 (vector風)
- キャッシュラインを意識したメモリレイアウト
- プリフェッチを活用した高速アクセス
- メモリプールとの連携

実装すべき機能

- 動的な要素追加・削除
- キャッシュ効率的な反復処理
- バルク操作 (一括挿入・削除)
- パフォーマンス測定機能

期待される出力例

```
=== キャッシュフレンドリーなデータ構造 ===

ベクター初期化（初期容量：16）
キャッシュライン境界でアラインメント済み

要素追加テスト：
1000要素を追加中...
容量自動拡張：16  32  64  128  256  512  1024

=== パフォーマンス比較 ===
順次アクセス（キャッシュ効率的）：

- キャッシュベクター：0.089ms
- 通常の配列：0.156ms
- 高速化：1.75倍

ランダムアクセス：

- キャッシュベクター：0.234ms
- 通常の配列：0.312ms
- 高速化：1.33倍

バルク操作（100要素一括挿入）：

- キャッシュベクター：0.012ms
- 個別挿入：0.089ms
- 高速化：7.42倍

=== キャッシュ統計 ===
L1キャッシュヒット率：94.3%
L2キャッシュヒット率：98.7%
プリフェッチ効果：23.4%の改善
```

ファイル名：ex15_6_cache_vector.c

問題15-7:プリプロセッサベースのDSL

プリプロセッサを使ってドメイン固有言語(DSL)を作成してください。

要件

- 状態機械を記述するDSL
- イベント駆動システムの記述
- 自動的なコード生成
- コンパイル時検証機能

実装例

```
STATE_MACHINE(TrafficLight)
  STATE(Red)    TIMEOUT() -> Yellow
  STATE(Yellow) TIMEOUT() -> Green
  STATE(Green)  TIMEOUT() -> Red
END_STATE_MACHINE
```

期待される出力例

```
=== プリプロセッサベースのDSL ===

信号機状態機械の定義:
状態数: 3
遷移数: 3

状態機械を実行中...
[00:00] 状態: Red
[00:05] タイムアウト → Yellow へ遷移
[00:06] 状態: Yellow
[00:08] タイムアウト → Green へ遷移
[00:09] 状態: Green
[00:14] タイムアウト → Red へ遷移
[00:15] 状態: Red

=== コード生成結果 ===
生成された関数:

- void TrafficLight_init()
- void TrafficLight_process_event(Event)
- State TrafficLight_get_state()
- const char* TrafficLight_state_name(State)

生成されたコード行数: 87行
マクロ展開前: 12行
圧縮率: 7.25倍

=== 検証結果 ===
すべての状態から遷移可能: PASS
デッドロックなし: PASS
未定義遷移なし: PASS
```

ファイル名: ex15_7_dsl.c

提出物

- ・ 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ・ 高度な言語機能や最適化を使用した箇所には、採用理由をコメントで記載してください。
- ・ 性能測定を求める問題には、測定条件と結果を含めてください。

学習のポイント

- ・ プリプロセッサは強力だが、使いすぎると可読性が低下する
- ・ メモリ管理の設計は、実行性能とメモリ使用量に影響する
- ・ キャッシュ効率を考慮すると大幅な性能向上が可能
- ・ アロケータやキャッシュなど、実行環境の仕組みを踏まえて設計する

第16章:C23の機能の演習問題

演習の目的

- C23標準で導入された新機能を理解する
- bool型、2進数リテラル、typeof演算子、nullptrを活用する
- モダンなCプログラミングスタイルを習得する
- より安全で表現力豊かなコードを書く

基礎問題

問題16-1:bool型の活用

bool型を使用して、簡単な状態管理システムを作成してください。

要件

- システムの各種状態をbool型で管理
- 状態の組み合わせによる判定ロジック
- 状態表示関数の実装
- 状態遷移の履歴記録

実装のヒント

```
typedef struct {
    bool is_connected;
    bool is_authenticated;
    bool has_permission;
    bool is_busy;
} SystemStatus;
```

期待される出力例

```
=== bool型を使った状態管理システム ===
```

```
初期状態:
接続: false
認証: false
権限: false
ビジー: false
```

```
システム接続中...
接続: true
認証: false (認証が必要です)
```

```
ユーザー認証...
接続: true
認証: true
権限: false (権限の付与待ち)
```

```
管理者権限を付与...
接続: true
認証: true
権限: true
ビジー: false
```

```
操作可能状態: Yes
```

```
=== 状態遷移履歴 ===
```

```
[00:00:00] システム起動
[00:00:01] 接続確立
[00:00:03] 認証成功
[00:00:05] 権限付与
[00:00:10] 処理開始 (ビジー: true)
[00:00:15] 処理完了 (ビジー: false)
```

問題16-2:ビット操作と2進数リテラル

2進数リテラルを使用して、8ビットのフラグ管理システムを作成してください。

要件

- 各ビットが異なる機能のON/OFFを表す
- ビットの設定、クリア、トグル、チェック関数
- 現在の状態を2進数で表示
- 複数フラグの一括操作

実装例

```
#define FLAG_READ    0b00000001
#define FLAG_WRITE   0b00000010
#define FLAG_EXECUTE 0b00000100
```

期待される出力例

```
=== 2進数リテラルを使ったフラグ管理 ===

権限フラグの定義:
FLAG_READ    = 0b00000001 (0x01)
FLAG_WRITE   = 0b00000010 (0x02)
FLAG_EXECUTE = 0b00000100 (0x04)
FLAG_DELETE  = 0b00001000 (0x08)

初期フラグ: 0b00000000

読み取り権限を設定:
現在のフラグ: 0b00000001 (READ)

書き込み権限を追加:
現在のフラグ: 0b00000011 (READ | WRITE)

すべての権限を設定:
現在のフラグ: 0b00001111 (READ | WRITE | EXECUTE | DELETE)

実行権限をトグル:
現在のフラグ: 0b00001011 (READ | WRITE | DELETE)

権限チェック:

- 読み取り: yes
- 書き込み: yes
- 実行: no
- 削除: yes

一括操作 (読み取り専用を設定):
現在のフラグ: 0b00000001 (READ)
```

応用問題

問題16-3:typeof演算子の応用

typeof演算子を使用して、汎用的なデータ構造操作マクロを作成してください。

要件

- 配列の最大値・最小値を求めるマクロ
- 2つの変数を安全に交換するマクロ
- 配列要素の合計を計算するマクロ
- 型安全な比較関数マクロ

実装例

```
#define ARRAY_MAX(arr, size) ({ \
    typeof(arr[0]) max = arr[0]; \
    /* 実装を完成させる */ \
    max; \
})
```

期待される出力例

```
=== typeof演算子の汎用マクロ ===

整数配列のテスト:
配列: [34, 67, 12, 89, 45, 23, 78, 56]
最大値: 89
最小値: 12
合計: 404

浮動小数点配列のテスト:
配列: [3.14, 2.71, 1.41, 1.73, 2.23]
最大値: 3.14
最小値: 1.41
合計: 11.22

安全な変数交換:
交換前: a=10, b=20
交換後: a=20, b=10

文字列の交換:
交換前: s1="Hello", s2="World"
交換後: s1="World", s2="Hello"

型安全な比較:
compare(10, 20) = -1
compare(3.14, 3.14) = 0
compare('Z', 'A') = 1
```

問題16-4:nullptr安全プログラミング

nullptrを活用して、安全なリンクリスト操作関数を作成してください。

要件

- ノードの追加、削除、検索
- すべてのポインター操作でnullptrチェック
- エラー処理の実装
- メモリリークの防止

実装のヒント

- 従来のNULLの代わりにnullptrを使用
- 戻り値でのエラー通知

期待される出力例

```
=== nullptr安全リンクリスト ===

要素を追加: 10
要素を追加: 20
要素を追加: 30

リストの内容: 10 -> 20 -> 30 -> nullptr

要素20を検索... 見つかりました
要素40を検索... 見つかりません

要素20を削除...
リストの内容: 10 -> 30 -> nullptr

空のリストに対する操作:
削除試行: エラー - リストが空です
検索試行: エラー - リストが空です

nullptrチェック統計:
- nullptrチェック総数: 42
- nullptr検出回数: 7
- 安全に処理された操作: 100%

メモリリーク検査: OK (すべてのメモリが解放されました)
```

問題16-5: デジット区切り記号の活用

デジット区切り記号を使って、大きな数値を扱うプログラムを作成してください。

要件

- 10進数、16進数、2進数での表記
- 可読性の高い定数定義
- ビットマスクの定義
- メモリサイズの計算

実装例

```
#define MEMORY_SIZE 16'777'216 // 16MB
#define MASK_32BIT 0xFFFF'FFFF
#define PATTERN 0b1010'1010'1010'1010
```

期待される出力例

```
=== デジット区切り記号を使った数値表現 ===

大きな数値の定義:
人口: 7'825'000'000 人
メモリサイズ: 16'777'216 バイト (16MB)
国家予算: 106'600'000'000'000 円

16進数表記:
アドレス空間: 0xFFFF'FFFF'FFFF'FFFF
カラーコード: 0x00FF'00FF (緑とマゼンタ)

2進数表記:
ビットパターン: 0b1010'1010'1010'1010
マスク: 0b1111'1111'0000'0000

計算例:
16MB = 16'777'216 バイト
      = 16'384 KB
      = 16 MB

ビット演算:
0b1111'0000'1111'0000 AND 0b1010'1010'1010'1010
= 0b1010'0000'1010'0000
```

チャレンジ問題

問題16-6:C23総合演習:設定管理システム

C23の複数の新機能を組み合わせた設定管理システムを作成してください。

要件

- bool型で各種設定のON/OFF管理
- 2進数リテラルでフラグ管理
- nullptrで安全なポインター操作
- typeof演算子でジェネリックな操作
- 設定の保存と読み込み機能

機能要件

- 設定項目の動的追加・削除
- 設定のグループ化
- デフォルト値の管理
- 設定変更の通知機能

期待される出力例

```
=== C23総合設定管理システム ===

設定グループ: Display

- brightness: true (bool)
- dark_mode: false (bool)
- resolution: 1920x1080
- refresh_rate: 60

設定グループ: Network

- wifi_enabled: true (bool)
- bluetooth_enabled: false (bool)
- airplane_mode: false (bool)

設定フラグ (2進数表示):
現在: 0b0000'0101 (WiFi | Brightness)

設定変更: dark_mode = true
通知: DisplaySettingsChanged -> dark_mode

設定の保存...
ファイル: settings.conf
保存完了 (チェックサム: 0xA5C3)

=== typeof演算子による汎用操作 ===
設定値の型安全な更新:

- 整数値: 60 -> 144 (成功)
- bool値: false -> true (成功)
- 文字列: "1920x1080" -> "2560x1440" (成功)
```

問題16-7:ビット演算デバグ

教育的なビット演算デバグを作成してください。

要件

- 2進数リテラルでの入力サポート
- ビット演算の可視化
- ステップ実行機能

- ・ 演算結果の詳細説明

実装すべき演算

- ・ AND, OR, XOR, NOT
- ・ ビットシフト(左右)
- ・ ビット回転
- ・ ビットカウント

期待される出力例

```

=== ビット演算デバッグ ===

入力方式を選択:

1. 2進数 (0b...)
2. 16進数 (0x...)
3. 10進数

選択: 1
値A: 0b1010'1100
値B: 0b1100'1010

=== ビット演算の可視化 ===

AND演算:
  1010 1100
& 1100 1010
-----
  1000 1000 (0x88)

OR演算:
  1010 1100
| 1100 1010
-----
  1110 1110 (0xEE)

XOR演算:
  1010 1100
^ 1100 1010
-----
  0110 0110 (0x66)

NOT演算:
~ 1010 1100
-----
  0101 0011 (0x53)

左シフト (2ビット):
1010 1100 << 2 = 1011 0000 (0xB0)
  [] 消失

右シフト (2ビット):
1010 1100 >> 2 = 0010 1011 (0x2B)
      消失 []

ビット数: A=5個, B=5個

```

提出物

- ・ 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ・ C23の機能を使用する箇所には、従来の書き方との差をコメントで記載してください。
- ・ 使用するコンパイラが対象機能を実装していることを確認してください。

学習のポイント

- ・ C23の新機能は従来のCコードをより安全で読みやすくする
- ・ bool型により意図が明確なコードが書ける

- 2進数リテラルはビット操作を直感的にする
- nullptrはポインターの安全性を向上させる
- typeof演算子により型安全なマクロが作成可能

補章2:開発環境の演習問題

演習の目的

開発環境の詳細な設定と診断、コンパイル過程の理解を深めます。

基礎問題

問題1:環境情報の確認

examples/environment_check.c を各C規格でコンパイルして実行し、結果を比較してください。

```
# C90準拠

gcc -std=c90 -pedantic environment_check.c -o check_c90
./check_c90

# C99準拠

gcc -std=c99 -pedantic environment_check.c -o check_c99
./check_c99

# C11準拠

gcc -std=c11 -pedantic environment_check.c -o check_c11
./check_c11
```

問題2:コンパイル過程の確認

以下のコマンドで各段階の出力を確認してください。

```
# プリプロセッサ出力

gcc -E examples/compiler_test.c > preprocessed.i

# アセンブリコード

gcc -S examples/compiler_test.c

# オブジェクトファイル

gcc -c examples/compiler_test.c

# 最終的な実行ファイル

gcc compiler_test.o -o compiler_test
```

応用問題

問題3:環境診断ツールの作成

solutions/env_diagnostic.c を作成し、以下の情報を表示するプログラムを作成してください。

表示項目

- コンパイラーの種類とバージョン
- C規格のバージョン
- システムのエンディアン
- 各データ型のサイズと範囲
- ポインターのサイズ(32bit/64bit判定)

問題4:コンパイラーオプションの比較

同じプログラムを異なるオプションでコンパイルし、違いを確認してください。

```
# 最適化なし

gcc -O0 test.c -o test_00

# 最適化レベル2

gcc -O2 test.c -o test_02

# サイズ最適化

gcc -Os test.c -o test_0s

# ファイルサイズを比較

ls -l test_*
```

チャレンジ問題

問題5:マルチ規格対応プログラム

異なるC規格で異なる動作をするプログラムを作成してください。

要求仕様

- プリプロセッサマクロで規格を判定
- 各規格特有の機能を使用
- 実行時に使用している規格を表示

問題6:包括的環境レポート生成ツール

開発環境の完全なレポートを生成するツールを作成してください。

機能要件

- HTMLまたはMarkdown形式でレポート出力
- 問題がある場合は警告と対処法を表示
- 推奨される設定を提案

参考資料

- [GCC公式ドキュメント](#)
- [Clang公式ドキュメント](#)
- [GNU Makeマニュアル](#)